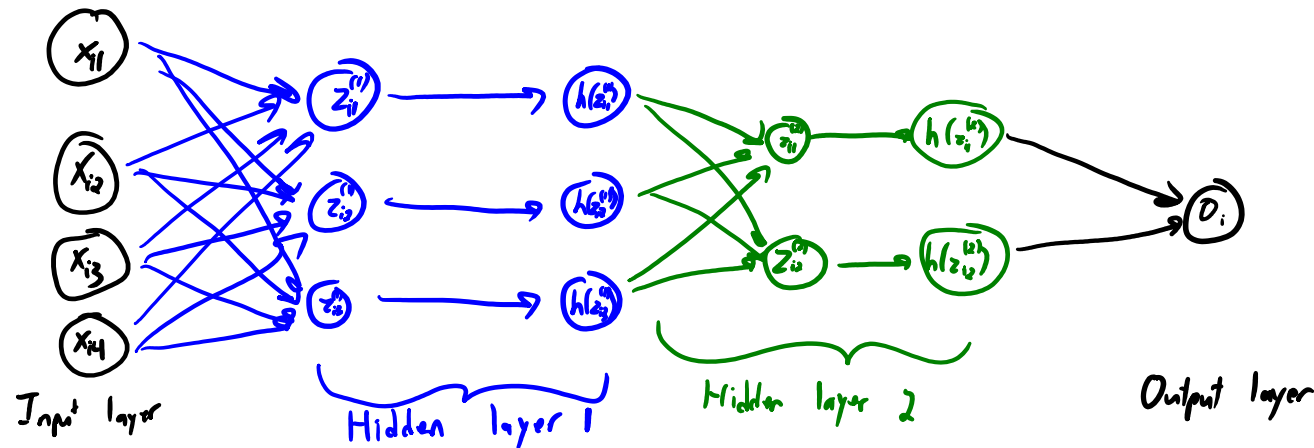


CPSC 340: Machine Learning and Data Mining

Deep Learning II

Last Time: Deep Learning

- Neural networks with multiple hidden layers:
 - We also add bias terms for each hidden neuron and each output neuron



$$o_i = v^T h(W^{(2)} \underbrace{h(W^{(1)} x_i)}_{z_i^{(1)}})$$

$z_i^{(2)}$

- Linear transformation followed by non-linear activation at each layer.
- Hidden layers learn latent features $h(z_i^{(l)})$; last activations serve as input to linear output layer.
 - Different objective functions lead to different tasks (e.g., binary classification, multi-class classification, regression).

Next Topic: Automatic Differentiation

Learning in Deep Neural Networks

- Usual training procedure is again **stochastic gradient descent (SGD)**.
 - Choose a random example 'i' (more common: mini-batch of samples).
 - Use **backpropagation** to get gradient with respect to all parameters.
 - Take a small step in the negative gradient direction.

- **Highly non-convex**, so are local minima the problem?
 - Empirical/theoretical evidence that **local minima are not the problem**.
 - We think all local optima are good in typical over-parameterized cases.
 - But can be **hard to get SGD close to a local minimum** in reasonable time.

- Challenging to make SGD work.
 - People have developed **tricks/modifications** to make it work.

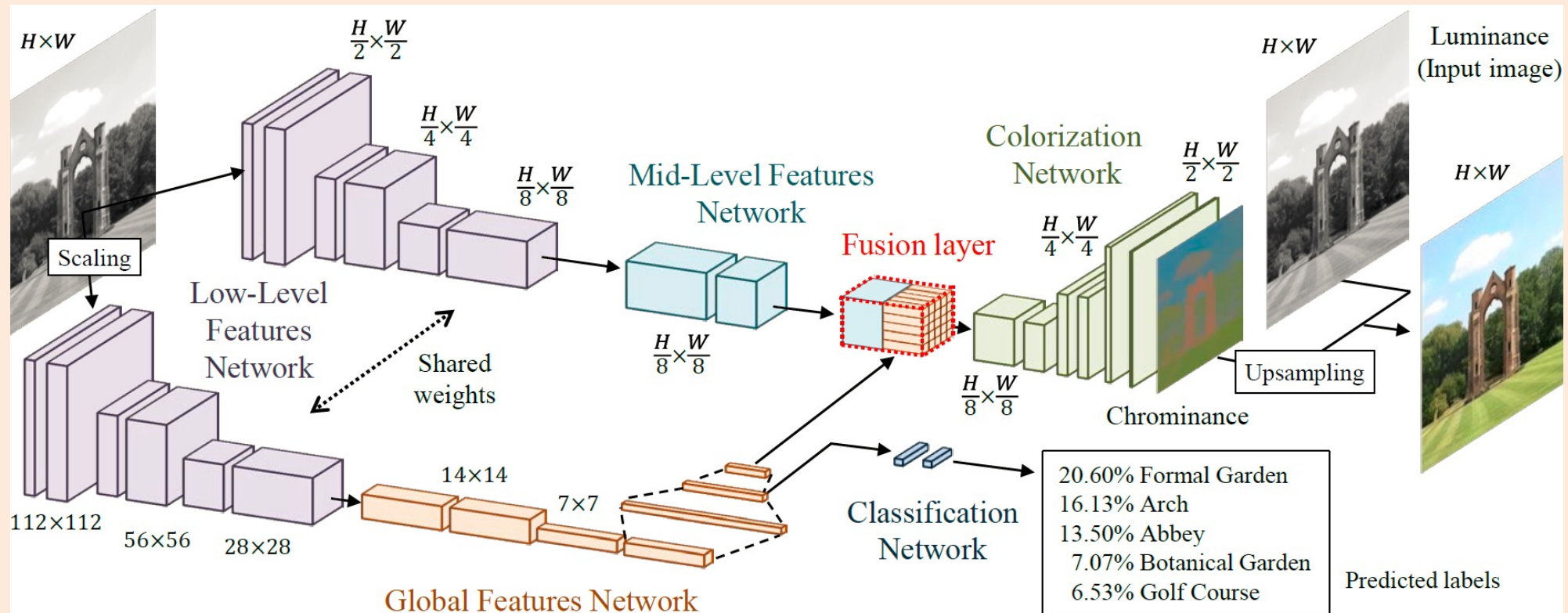
It is different
from linear
model.

Things that
works

Empirical Again!

More-Complicated Layers

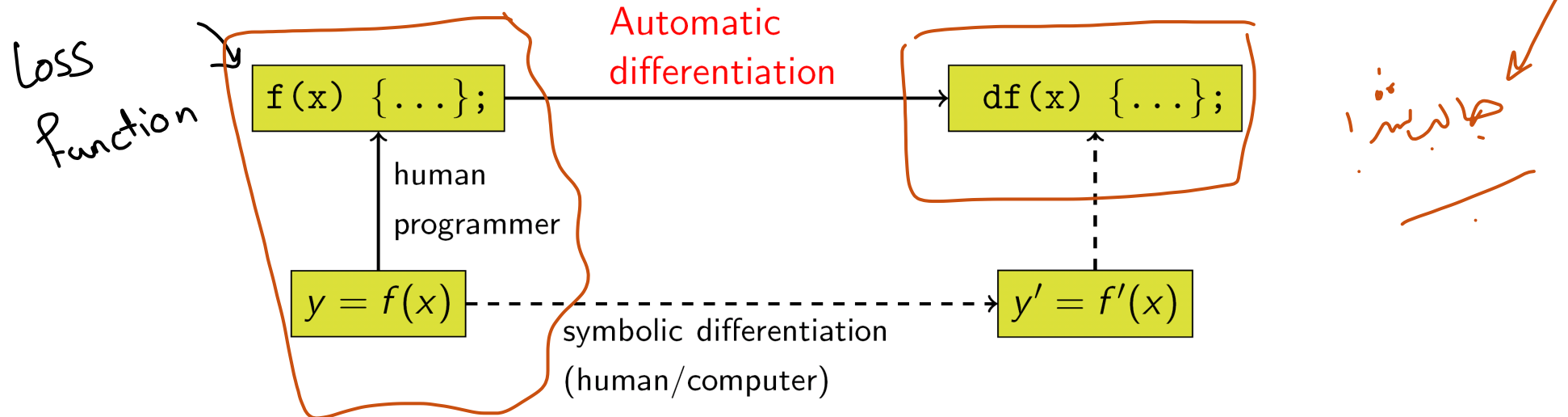
- Modern networks often have complicated structures:
 - Each step might be doing a different operation.
 - This makes **coding up the gradient both time-consuming and prone to errors.**



- Developing networks like this is made easier using **automatic differentiation**.

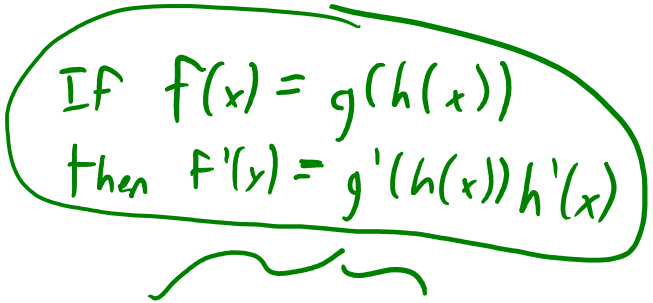
Automatic Differentiation (AD)

- Automatic differentiation (AD):
 - Input: code computing a function.
 - Output: code to compute **one or more derivatives** of the function.
 - **No loss in accuracy**, unlike finite-difference approximations. $\rightarrow$ exactly gradient
 - The output code has the **same asymptotic runtime** as the input code.
 - Does not give you a “formula” for the derivative, **just code that computes it.**



“Reverse Mode” Automatic Differentiation (AD)

- In machine learning, we typically use “reverse mode” AD.
 - Gives code for computing the gradient of a differentiable function.
 - The slides will exclusively talk about “reverse mode”. For “forward mode”, see bonus.
 - You use this gradient to train deep learning models via SGD.
- Has a close connection to backpropagation.
 - Classic algorithm to compute the gradient of neural network parameters.
 - “Apply the chain rule, store redundant calculations”.
 - Backpropagation can be viewed as special case of AD.
- AD basically writes every operation as instance of the chain rule.


$$\text{If } f(x) = g(h(x)) \\ \text{then } f'(x) = g'(h(x))h'(x)$$

Automatic Differentiation – Single Input+Output

- Consider the function $f(x) = 10 * \log(1 + \exp(-2 * x))$.
- We write the function as a **series of compositions**: $f_5(f_4(f_3(f_2(f_1(x)))))$.
 - Where $f_1(z) = -2 * z$, $f_2(z) = \exp(z)$, $f_3(z) = 1 + z$, $f_4(z) = \log(z)$, $f_5(z) = 10 * z$.
 - So we have $f_1'(z) = -2$, $f_2'(z) = \exp(z)$, $f_3'(z) = 1$, $f_4'(z) = 1/z$, $f_5'(z) = 10$.
 - These all cost $O(1)$.
- Recursively applying the chain rule we get:

$$f'(x) = f_5'(f_4(f_3(f_2(f_1(x)))) * f_4'(f_3(f_2(f_1(x)))) * f_3'(f_2(f_1(x))) * f_2'(f_1(x)) * f_1'(x).$$



$$\underbrace{10}_{f_5'} * \underbrace{1/f_3(f_2(f_1(x)))}_{f_4'} * \underbrace{1}_{f_3'} * \underbrace{\exp(f_1(x))}_{f_2'} * \underbrace{-2}_{f_1'} \Rightarrow$$

$$\frac{1}{1 + \exp(-2x)}$$

$$\frac{-2 \exp(-2x)}{1 + \exp(-2x)}$$

Handwritten note: *جواب سہ !* (Answer is 3 !)

Automatic Differentiation – Single Input+Output

- Our function written as a set of compositions:
 - $f_5(f_4(f_3(f_2(f_1(x)))))$.
- The derivative written using the chain rule:
 - $f'(x) = f_5'(f_4(f_3(f_2(f_1(x)))))*f_4'(f_3(f_2(f_1(x))))*f_3'(f_2(f_1(x)))*f_2'(f_1(x))*f_1'(x)$.
- Notice that this leads to **repeated calculations**.
 - For example, we use $f_1(x)$ four different times.
 - We can **use dynamic programming to** avoid redundant calculations.
- First, the “**forward pass**” will compute and store the expressions:
 - $\alpha_1 = f_1(x), \alpha_2 = f_2(\alpha_1), \alpha_3 = f_3(\alpha_2), \alpha_4 = f_4(\alpha_3), \alpha_5 = f_5(\alpha_4) = f(x)$. *→ you calculate all of this*
- Next, the “**backward pass**” uses stored α_k values and f_i' functions: *For your loss function*
 - $\beta_5 = 1*f_5'(\alpha_4), \beta_4 = \beta_5*f_4'(\alpha_3), \beta_3 = \beta_4*f_3'(\alpha_2), \beta_2 = \beta_3*f_2'(\alpha_1), \beta_1 = \beta_2*f_1'(x) = f'(x)$.
- A generic method to make **code computing $f'(x)$ for same cost as $f(x)$** . *Trade-off between computing power vs. memory.*

Automatic Differentiation – Multiple Parameters

- In ML problems, we often have **more than 1 parameter**.
 - And we **want to compute the gradient** for the same cost as the function.
- To generalize AD to this case, we define a **computation graph**:
 - A directed acyclic graph (DAG).
 - **Root nodes are the parameters (and inputs).**
 - **Intermediate nodes are computed values (α values).**
 - **Leaf node is the function value.**
- Computing the gradient with AD:
 - The **forward pass** evaluates the function and stores intermediate values.
 - Going from the roots through the intermediate nodes to the leaf.
 - The **backward pass** applies the f_i' functions to the α values.
 - Accumulating the needed pieces of the chain rule until **each root has its partial derivative.**

Example

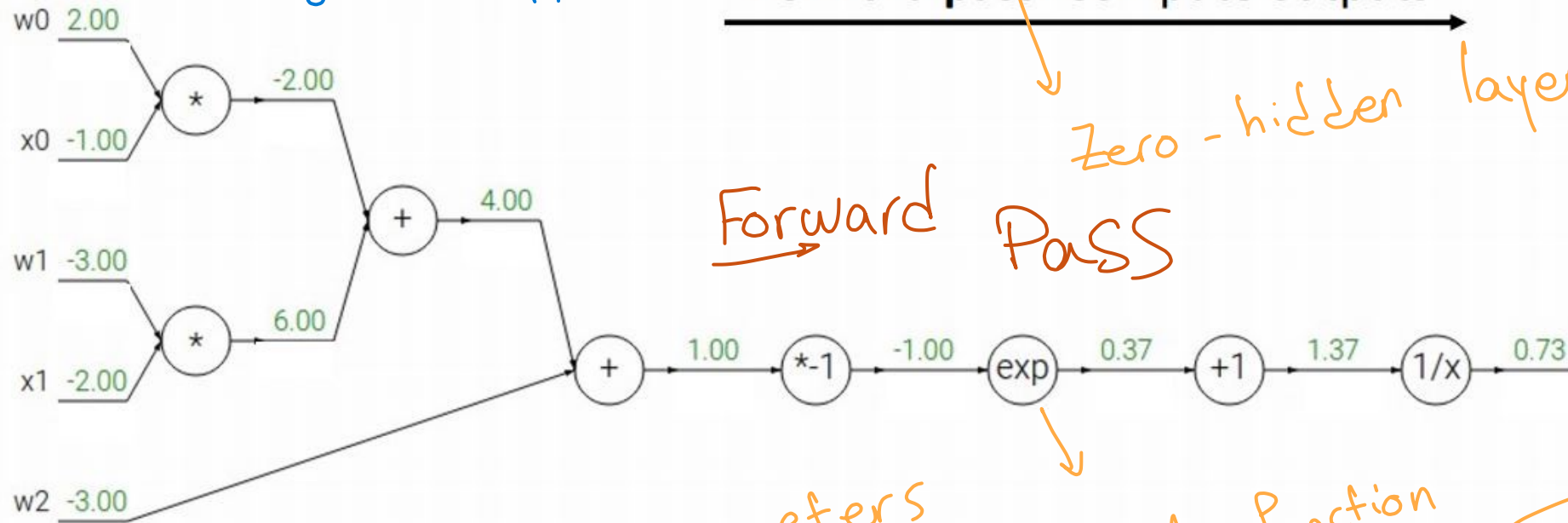
Another Example

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

Logistic
Regression

2 dimensions (x_0 and x_1)

Forward pass: Compute outputs



Forward
Pass

Zero-hidden layer.

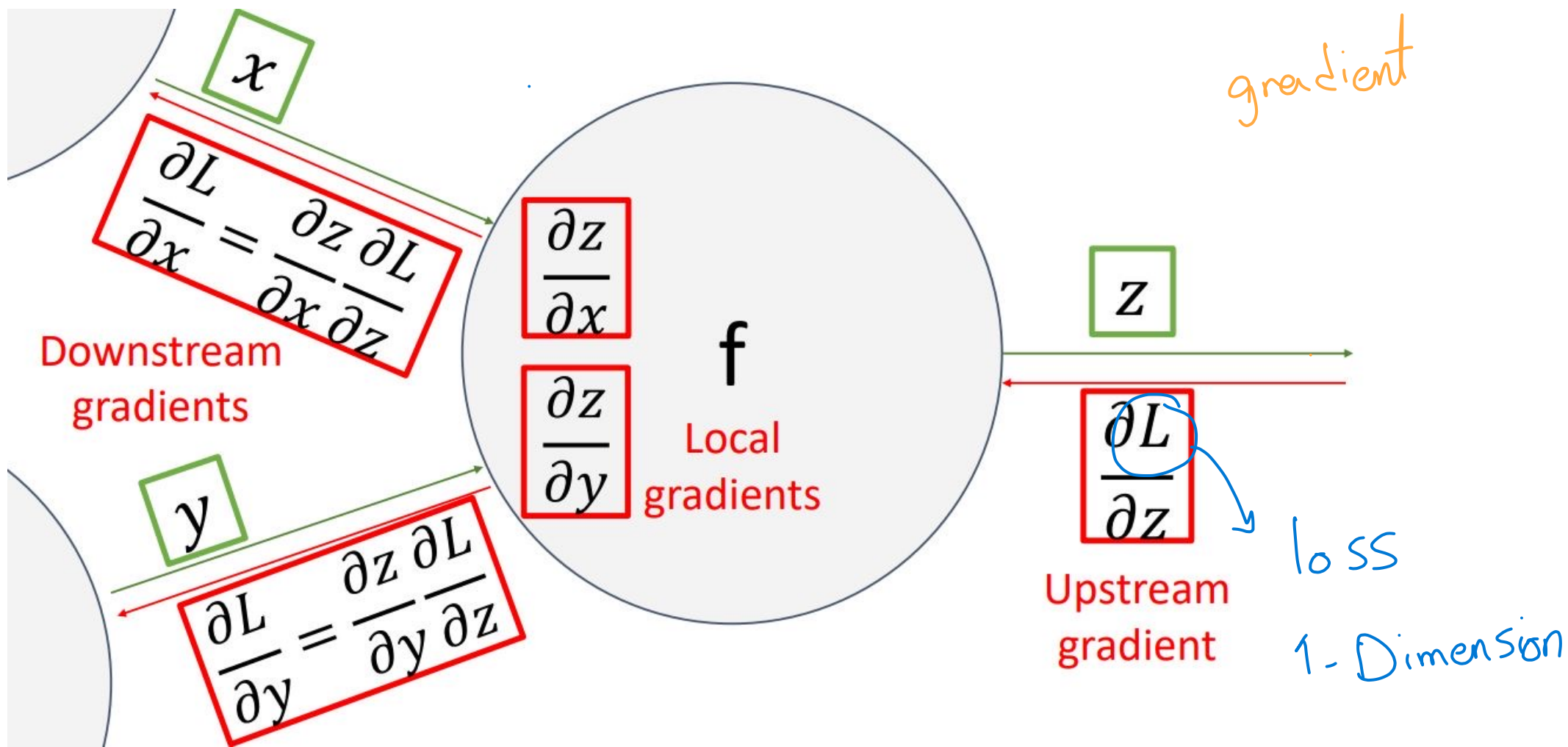
+ w_2 bias

3 parameters

Each Function
Comes with a
gradient.

Video from former UBC master's student Andrej Karpathy (of OpenAI, former director of AI and Autopilot Vision at Tesla)

Example

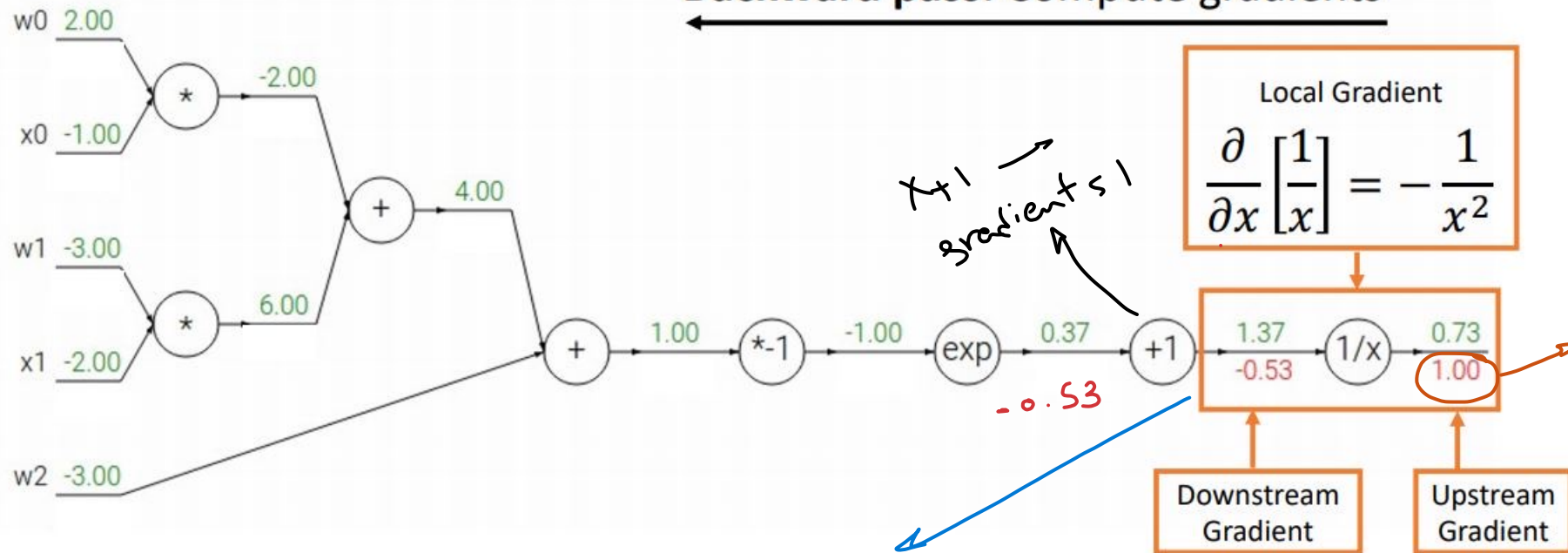


Example

Another Example $f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$

Goal: Find gradient wrt w_0, w_1 , and w_2

Backward pass: Compute gradients

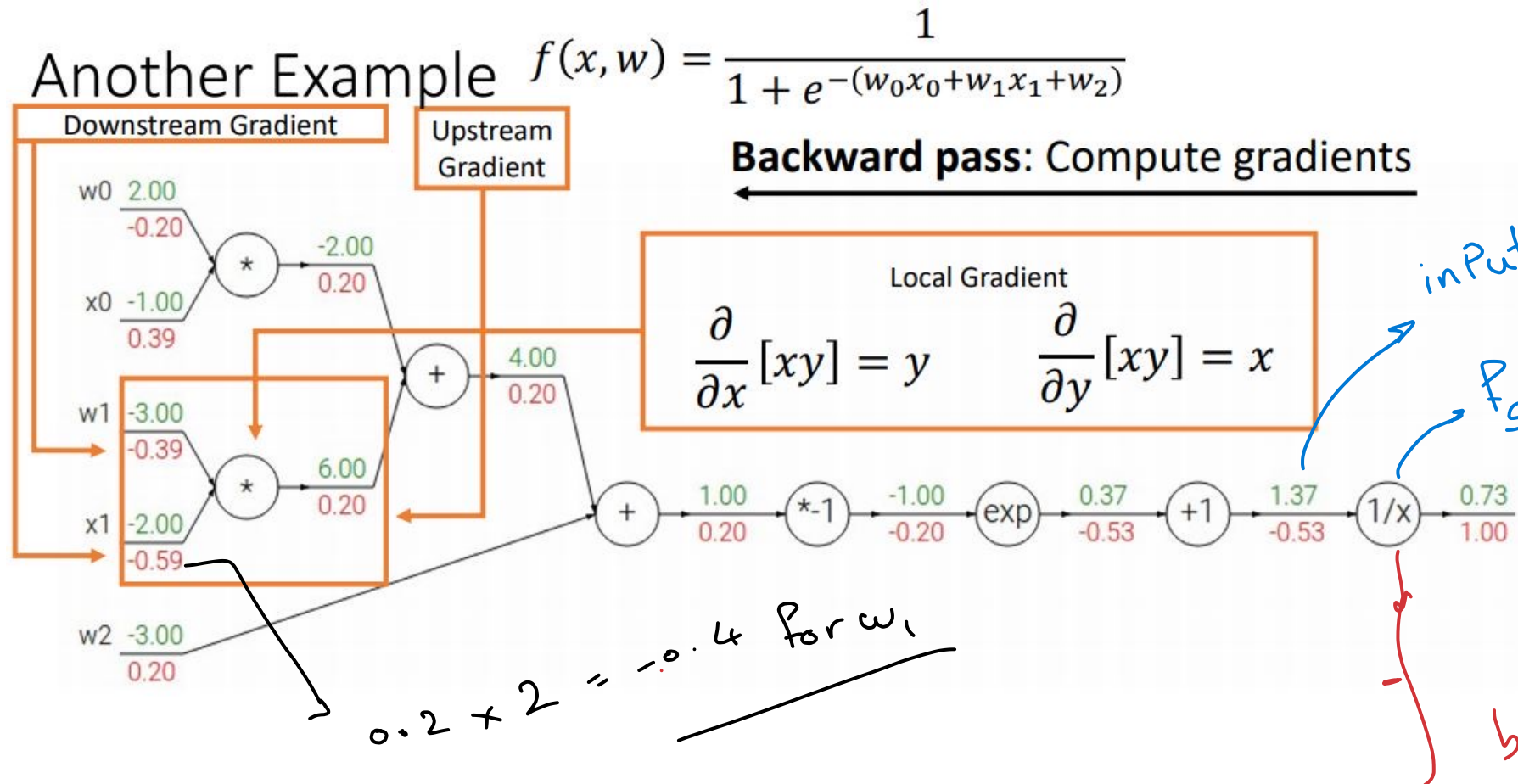


$$P'_S = -\frac{1}{x^2}, \quad \frac{-1}{(1.37)^2} = -0.53$$

$$P'_S = -\frac{1}{x^2}$$

Start with this

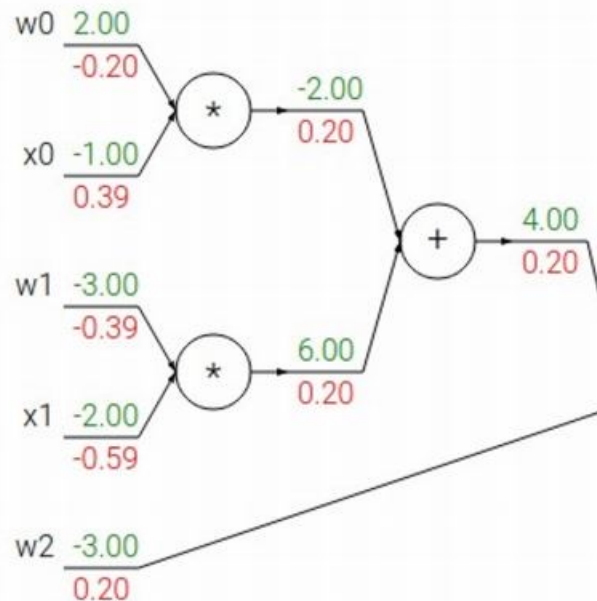
Example



Example

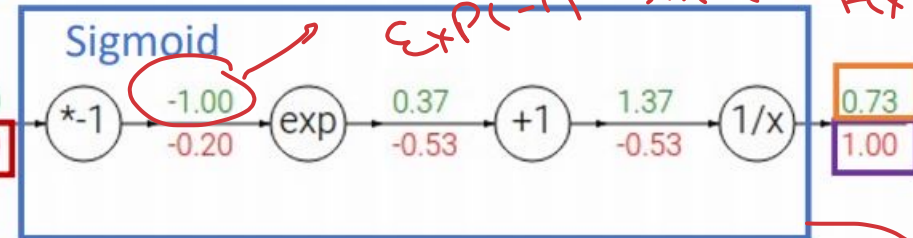
Another Example $f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}} = \sigma(w_0x_0 + w_1x_1 + w_2)$

Backward pass: Compute gradients



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Computational graph is not unique: we can use primitives that have simple local gradients



$$[\text{Downstream}] = [\text{Local}] * [\text{Upstream}]$$

$$= (1 - 0.73) * 0.73 * 1.0 = 0.2$$

Sigmoid local gradient: $\frac{\partial}{\partial x} [\sigma(x)] = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$

Vector Form

Sigmoid doesn't
have to be
decomposed oo

4D input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$f(x) = \max(0, x)$
(elementwise)

4D output y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

4D dL/dx:

$\begin{bmatrix} 4 \\ 0 \\ 5 \\ 0 \end{bmatrix}$

$\begin{bmatrix} dy/dx & dL/dy \end{bmatrix}$

$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

4D dL/dy:

$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

Upstream
gradient

Automatic Differentiation – Neural Networks

- Computing gradient for neural networks is a **special case of automatic differentiation**:

- Forward AD pass is called **forward propagation**:

- Starts from x_i and processes layers to reach $\hat{y}_i$.
 - Storing intermediate calculations.

- Backward AD pass is called **backpropagation**:

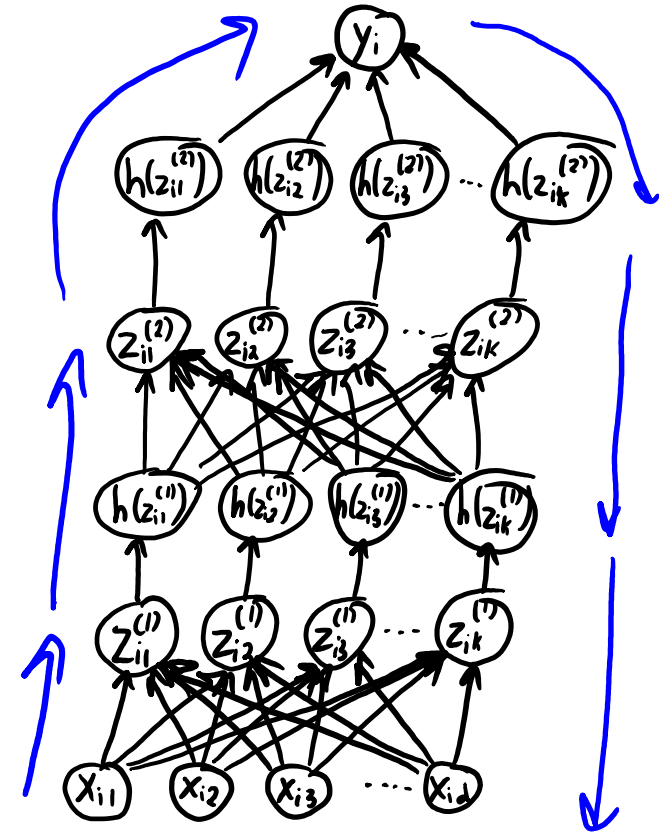
- Computes gradient of last layers and works backwards.
 - Using intermediate calculations stored during forward pass.

- Do you need to know how to do this?

- Exact details are probably not vital (exist many implementations).
- But understanding basic idea helps you **know what can go wrong**.
 - Or **give hints about what to do when you run out of memory**.

PyTorch, oo

- Backward pass has same cost as forward pass**.
 - So cost of computing gradient is same as cost of making predictions.



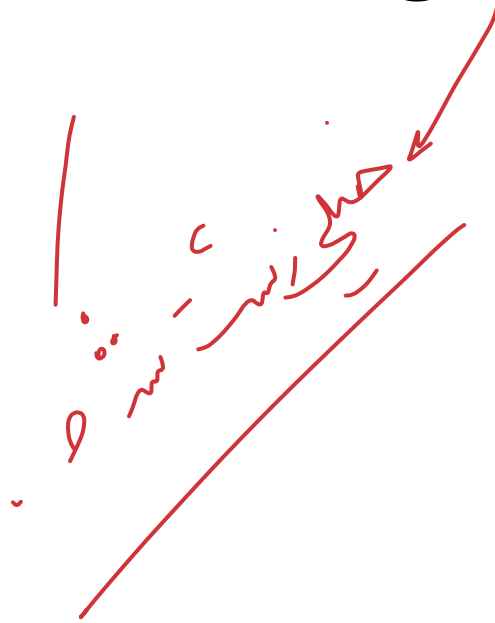
Automatic Differentiation - Discussion

- **AD is amazing** – get gradient for the same cost as the function.
 - You can try out lots of stuff, and enjoy thoroughly overfitting validation set!
 - Modern AD codes have lots of features, like built-in derivatives of matrix operations.
- But reverse-mode AD has some drawbacks:
 - Need to **store all intermediate calculations**, so requires a lot of storage.
 - For basic deep neural networks, hand-written code would only need to store the activations.
 - Modern codes have some of these space savings built in.
 - For other functions, the storage cost of AD is much higher than handwritten derivative code.
 - “Checkpointing” exists to reduce storage, but increases computational cost.
 - Has the **same cost as computing the function**, which is a pro and a con.
 - For basic deep neural networks, these have the same cost so this is what we want.
 - For other functions, the gradient can be computed at a lower cost than the function value.
 - May **miss opportunities for parallelism**, or **miss tricks to avoid numerical problems**.
- **AD only makes sense at points where the function is differentiable.**
 - TensorFlow and PyTorch can give incorrect “subderivatives” at non-differentiable ReLU points.
 - AD cannot do things like “take the derivative of a function of a sample from the distribution”.

Conceptual Questions

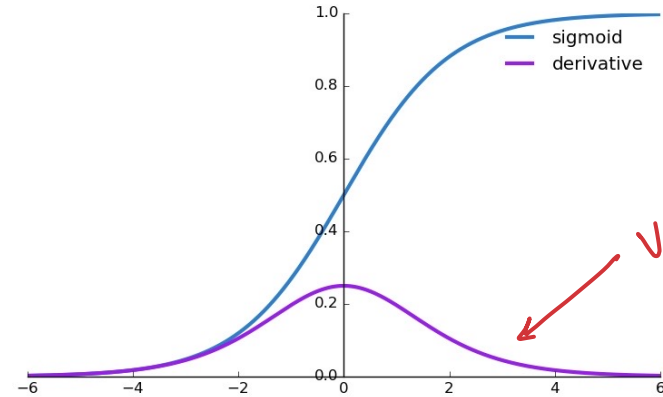
- Which of the followings are true about AD:
- A: AD calculates numerical approximation of derivatives
- B: AD helps train neural networks by efficiently computing gradients
- C: Backpropagation is an application of reverse-mode AD for computing gradients in neural networks
- D: Reverse-mode AD computes derivatives for all inputs in a single backward pass

Next Topic: Vanishing Gradients

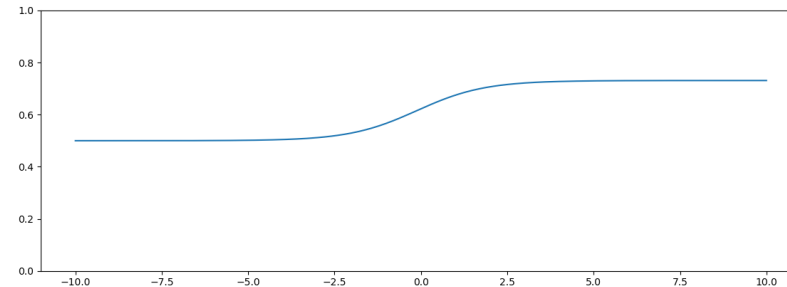


New Issue with DL: Vanishing Gradients

- Consider the sigmoid function:



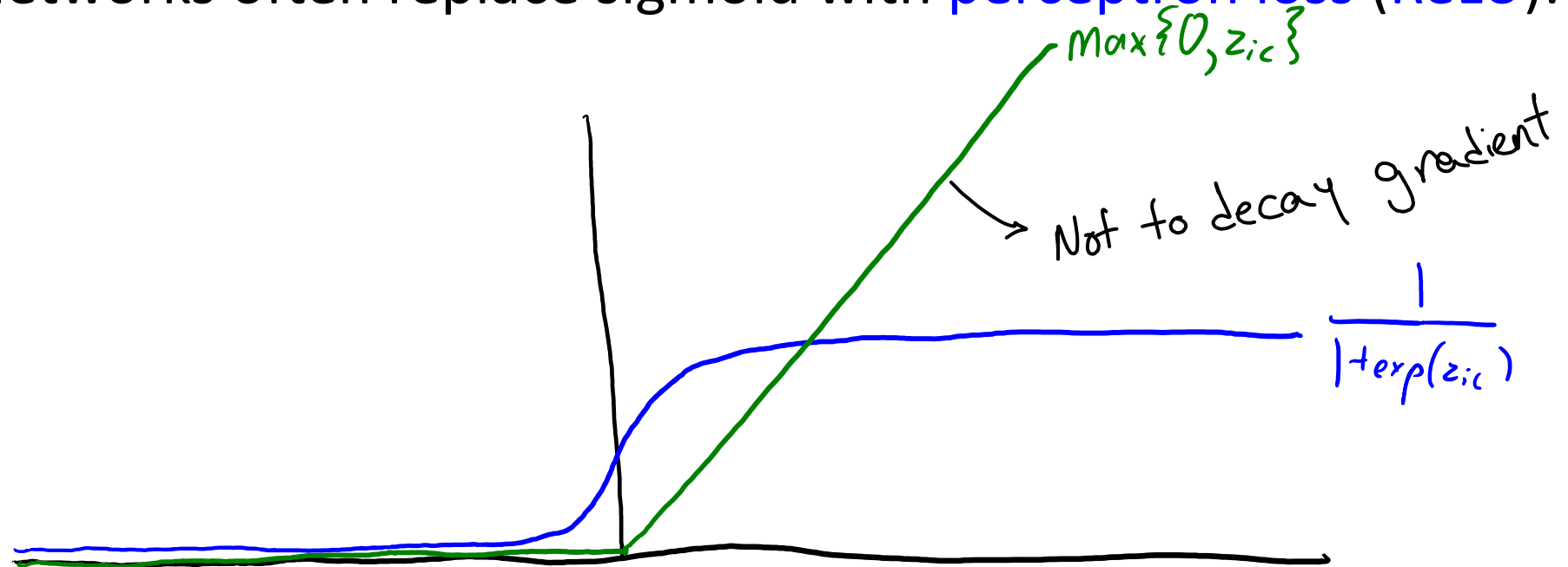
- Away from the origin, the **gradient is nearly zero**.
- The problem gets worse when you take the **sigmoid of a sigmoid**:



- In deep networks, many **gradients can be nearly zero everywhere**.
 - And numerically they will be set to 0, so **SGD does not move**.

Rectified Linear Units (ReLU)

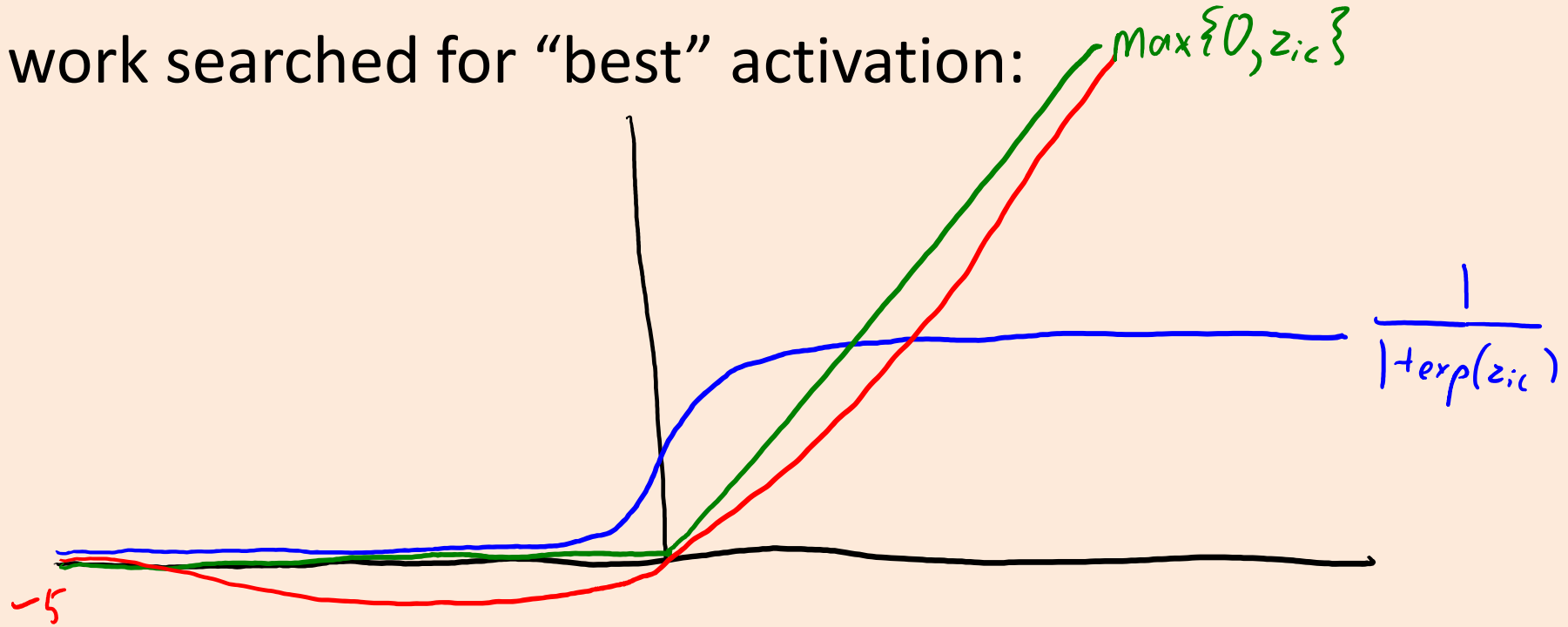
- Modern networks often replace sigmoid with **perceptron loss (ReLU)**:



- Just **sets negative values z_{ic} to zero**.
 - Reduces vanishing gradient problem (positive region is never flat).
 - Gives sparser activations.
 - Still **gives a universal approximator** if size of hidden layers grows with 'n'.

“Swish” Activation

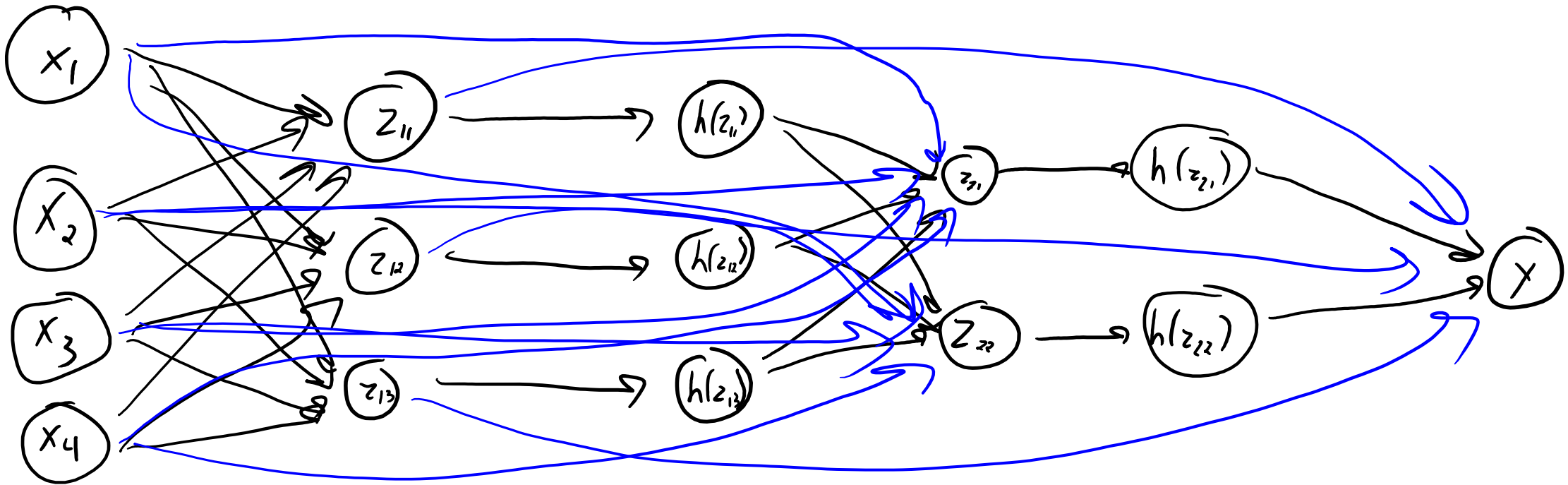
- Recent work searched for “best” activation:



- Found that $z_{ic}/(1+\exp(-z_{ic}))$ worked best (“swish” function).
 - A bit weird because it allows negative values and is non-monotonic.
 - But basically the same as ReLU when not close to 0.

Skip Connections Deep Learning

- Skip connections can also reduce vanishing gradient problem:



- Makes “shortcuts” between layers (so fewer transformations).
 - Many variations exist on skip connections exist.

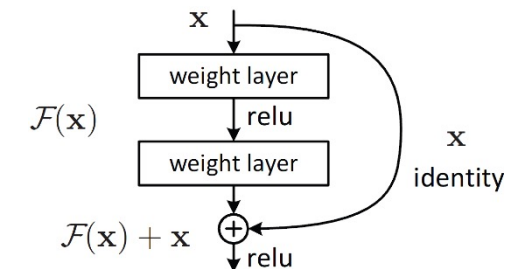
ResNet “Blocks”

- **Residual networks (ResNets)** are a variant on skip connections.
 - Consist of repeated “blocks”, first methods that successfully used 100+ layers.
- Usual computation of activation based on previous 2 layers:

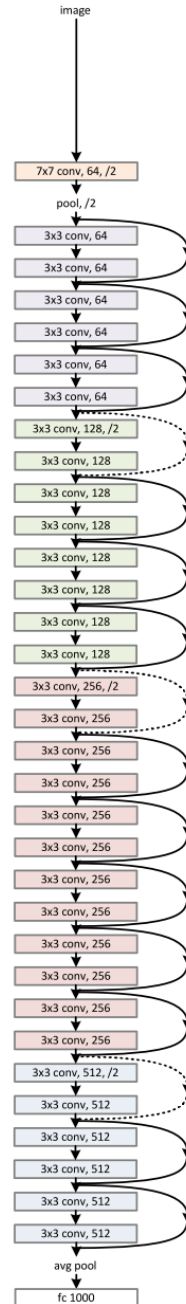
$$a^{p+2} = h(W^{p+1} h(W^p a^p))$$

↑ “activation” at layer ‘p’

- ResNet “**block**”: $a^{p+2} = h(a^p + W^{p+1} h(W^p a^p))$
 - Adds activations from “2 layers ago”.
- Differences from usual skip connections:
 - Activations vectors a^p and a^{p+2} must have the **same size**.
 - **No weights on a^p** , so W^p and W^{p+1} must focus on “updating” a^p (fit “residual”).
 - If you use ReLU, then $W^p=0$ implies $a^{p+2}=a^p$ so “free” information flows across layers.



34-layer residual



Common Deep Learning Tricks

- Data standardization (“centering” and “whitening”).
- Parameter initialization: “small but different”.
 - If we initialize all parameters in the layer to same value, they stay the same.
 - Also common to use initializations that are **standardized within layers**.
- Step-size selection: “babysitting”.
 - Use bigger step-size for the bias variables, or different for each layer.
 - Methods that use a step size for each coordinate (AdaGrad, RMSprop, Adam).
- **Early stopping** of the optimization based on validation accuracy.
- **Momentum**: adds weighted sum of previous SGD directions.
- **Batch normalization**: adaptive standardizing within layers.
 - Often allows sigmoid activations in deep networks.

Common Deep Learning Tricks

- L2-regularization or L1-regularization (“weight decay”).
 - Sometimes with different λ for each layer.
 - Recent work shows this can introduce bad local optima.
- Dropout: randomly zeroes activations ‘z’ values to discourage dependence.
- Rectified linear units (ReLU) as non-linear transformation.
 - Makes objective non-differentiable, but we now know SGD still converges in this setting.
- Residual/skip connections: connect layers to multiple previous layers.
 - We now know that such connections make it more likely to converge to good minima.
- Neural architecture search: try to cleverly search space of hyper-parameters.
 - This gets expensive!
- Some of these tricks are explored in bonus slides.

Missing Theory Behind Training Deep Networks

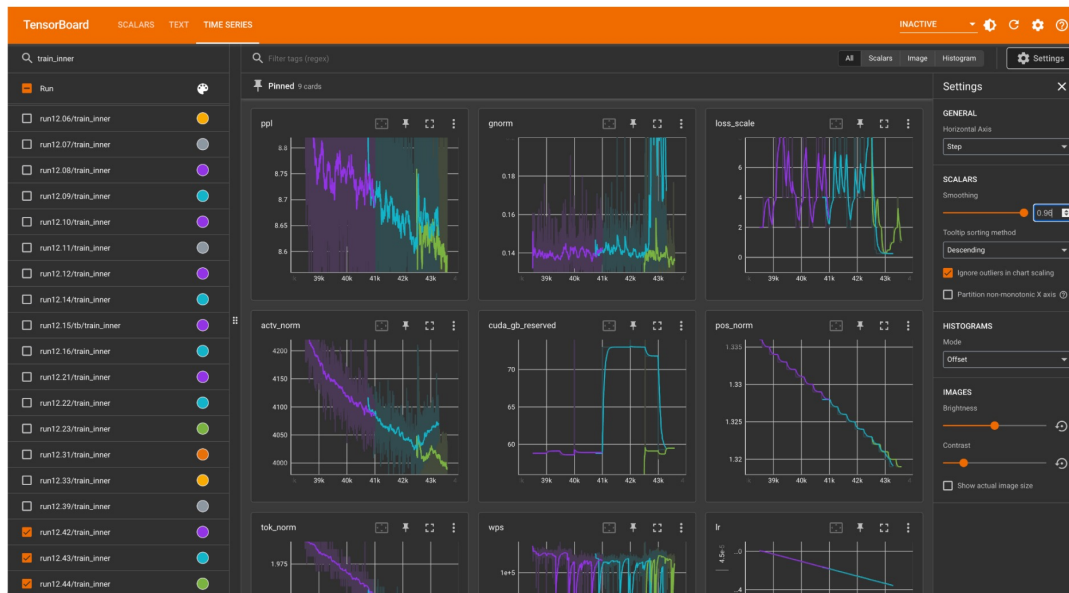
- Unfortunately, we **do not understand many of these tricks** very well.
 - Large portion of theory is on degenerate case of linear neural networks.
 - Or other weird cases like “1 hidden unit per layer”.
 - A lot of research is performed using “**grad student descent**”.
 - Several variations are tried, ones that perform well empirically are kept (possibly overfitting).
- Popular Examples:
 - **Batch normalization** originally proposed to fix “internal covariate shift”.
 - Internal covariate shift not defined in original paper, and batch norm doesn’t seem to reduce it.
 - Often singled out as an **example of problems with machine learning scholarship**.
 - Like many heuristics, people use batch norm because they found that it often helps.
 - Many people have worked on better explanations.
 - **Adam optimizer** is a nice combinations of ideas from several existing algorithms.
 - Such as “momentum” and “AdaGrad”, both of which are well-understood theoretically.
 - But theory in the original paper was incorrect, and Adam fails at solving some very-simple optimization problems.
 - But Adam is often used because it is amazing at training some networks.
 - It has been hypothesized that we “**converged**” towards networks that are easier for current SGD methods like Adam.

Modern Babysitting

- From 114-page babysitting log of training 175B parameter model:

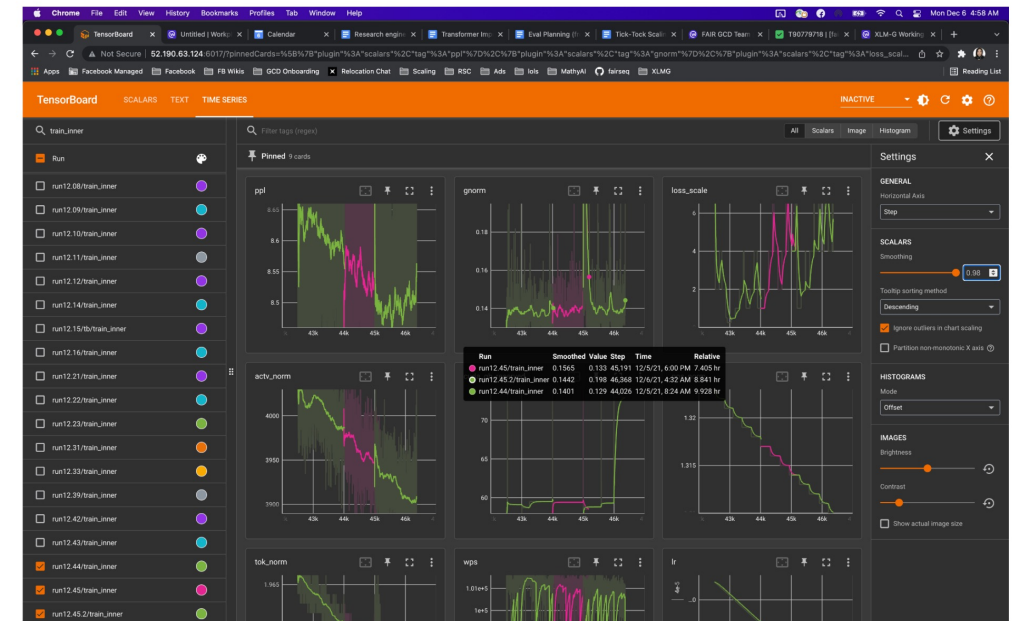
2021-12-05 05:35 PT: Checking on 12.44

[From Susan: looks like lowering LR helps keep us on track wrt ppl. Loss scale that stays below 1 for too long could be a leading indicator of instability going forward. Set smoothing to ~0.95 to see these trends.]



2021-12-06 05:00 PT: Grad norm spiking, ppl trending up

[From Susan: reading more tea leaves here, but seems like we've had a couple of grad norm spikes and our ppl is now slowly diverging. Recommending restarting with half the LR]



Digression: Deep Learning Vocabulary

- “Deep learning”: Models with many hidden layers.
 - Usually neural networks.
- “Neuron”: node in the neural network graph.
 - “Visible unit”: feature.
 - “Hidden unit”: latent factor z_{ic} or $h(z_{ic})$.
- “Activation function”: non-linear transform.
- “Activation”: $h(z_i)$.
- “Backpropagation”: compute gradient of neural network.
 - Sometimes “backpropagation” means “training with SGD”.
- “Weight decay”: L2-regularization.
- “Cross entropy”: softmax loss.
- “Learning rate”: SGD step-size.
- “Learning rate decay”: using decreasing step-sizes.
- “Vanishing gradient”: underflow/overflow during gradient calculation.

Summary

- Automatic differentiation:
 - Decomposing code using the chain rule, to make derivative code.
- Vanishing gradient in deep networks (gradient may be close to 0).
 - Can be reduced using rectified linear units (ReLU) as non-linear transform.
 - Can be reduced using various forms of skip connections.
- ResNets include untransformed previous layers.
 - Network focuses non-linearity on residual, allows huge number of layers.
- Overview of tricks for training deep networks.
 - Theory is poorly understood for many of these tricks.
- Next time: The most important idea in computer vision?

Parameter Initialization

- **Parameter initialization** is crucial:
 - Can't initialize weights in same layer to same value, or they will stay same.
 - Can't initialize weights too large, it will take too long to learn.
- A traditional **random initialization**:
 - Initialize bias variables to 0.
 - **Sample** from standard normal, divided by 10^5 ($0.00001 * \text{randn}$).
 - $w = .00001 * \text{randn}(k, 1)$
 - Performing multiple initializations does not seem to be important.

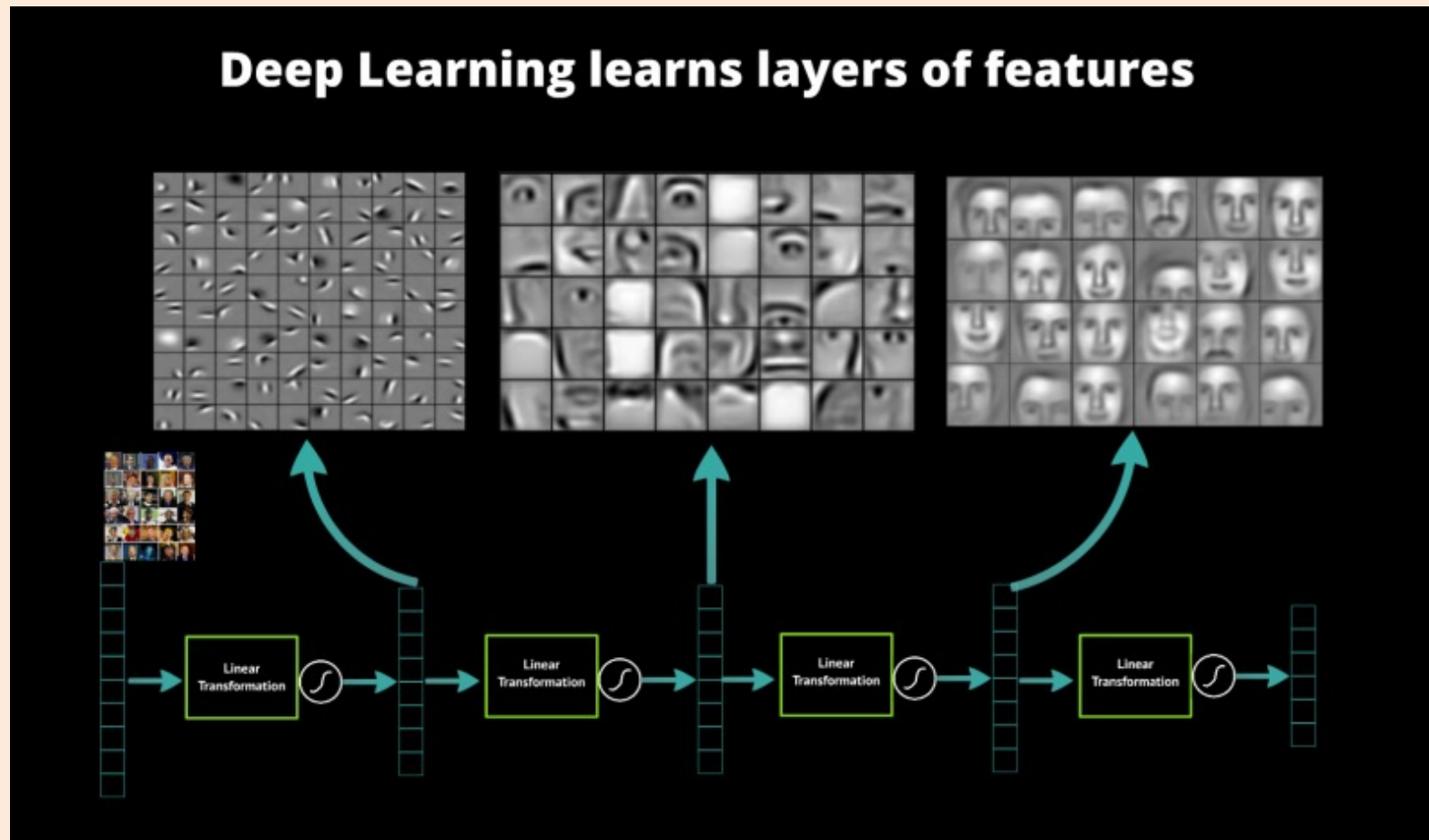
Parameter Initialization

- **Parameter initialization** is crucial:
 - Can't initialize weights in same layer to same value, or they will stay same.
 - Can't initialize weights too large, it will take too long to learn.
- Also common to **transform data** in various ways:
 - Subtract mean, divide by standard deviation, “whitten”, standardize y_i .
- More recent initializations try to **standardize initial z_i** :
 - Use **different initialization in each layer**.
 - Try to **make variance of z_i the same across layers**.
 - Popular approach is to sample from standard normal, divide by $\sqrt{2 * n_{\text{Inputs}}}$.
 - Use samples from uniform distribution on $[-b, b]$, where

$$b = \frac{\sqrt{6}}{\sqrt{k^{(m)} + k^{(m-1)}}}$$

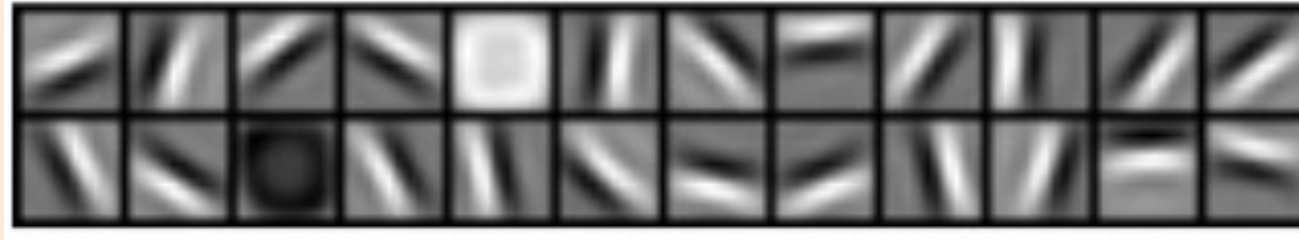
Cool Picture Motivation for Deep Learning

- Faces might be composed of different “parts”:



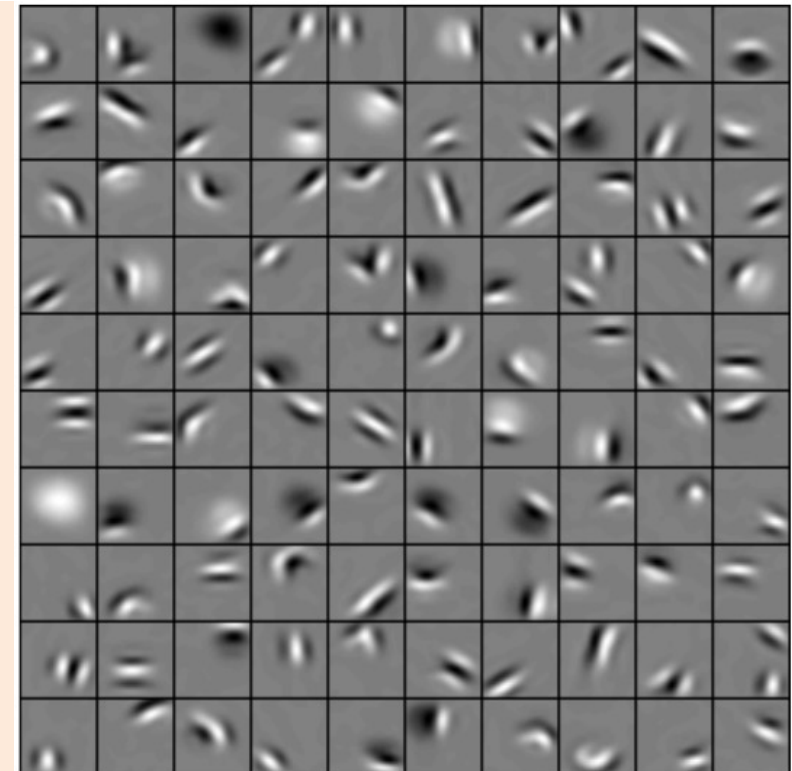
Cool Picture Motivation for Deep Learning

- First layer of z_i trained on 10 by 10 image patches:



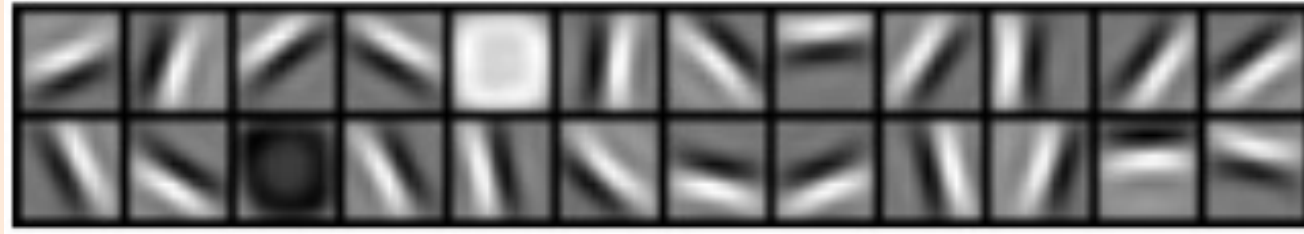
} "Gabor filters"

- Attempt to visualize second layer:
 - Corners, angles, surface boundaries?
- Models require many tricks to work.
 - We'll discuss these next time.



Cool Picture Motivation for Deep Learning

- First layer of z_i trained on 10 by 10 image patches:



} "Gabor filters"

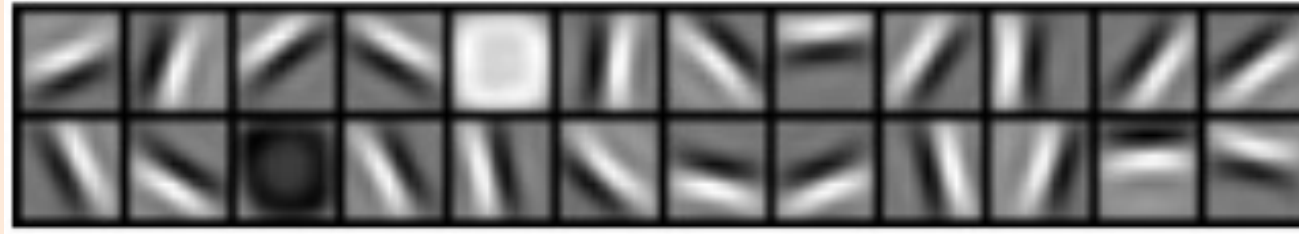
- Visualization of second and third layers trained on specific objects:

faces



Cool Picture Motivation for Deep Learning

- First layer of z_i trained on 10 by 10 image patches:



} "Gabor filters"

- Visualization of second and third layers trained on specific objects:

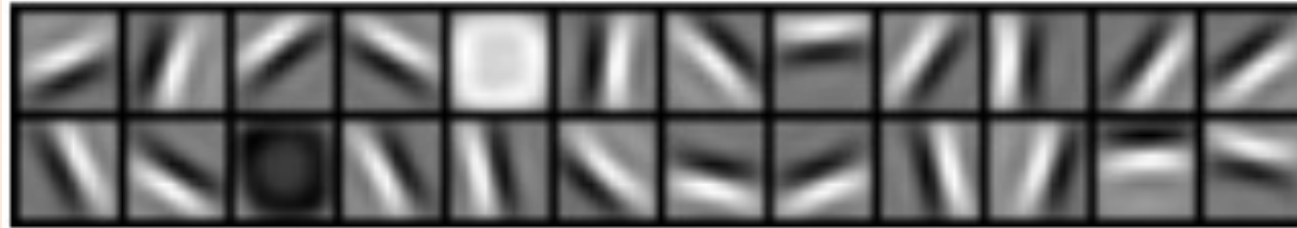
faces

cars



Cool Picture Motivation for Deep Learning

- First layer of z_i trained on 10 by 10 image patches:



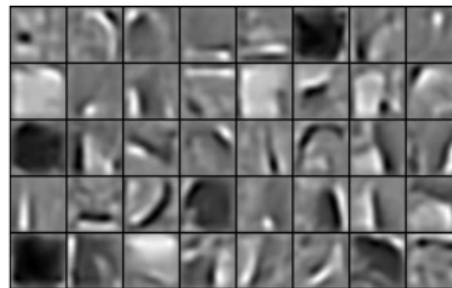
} "Gabor filters"

- Visualization of second and third layers trained on specific objects:

faces

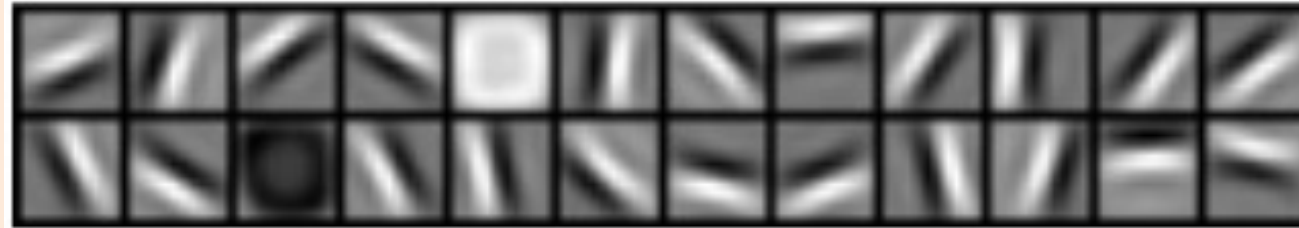
cars

elephants



Cool Picture Motivation for Deep Learning

- First layer of z_i trained on 10 by 10 image patches:



} "Gabor filters"

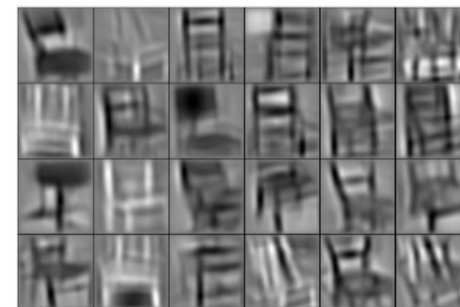
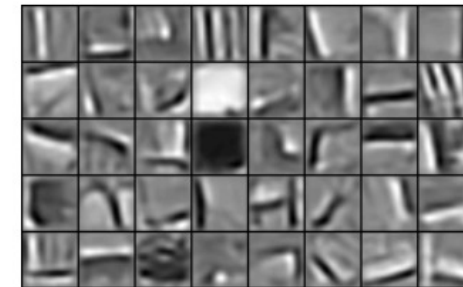
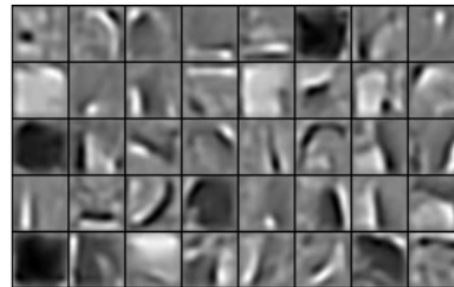
- Visualization of second and third layers trained on specific objects:

faces

cars

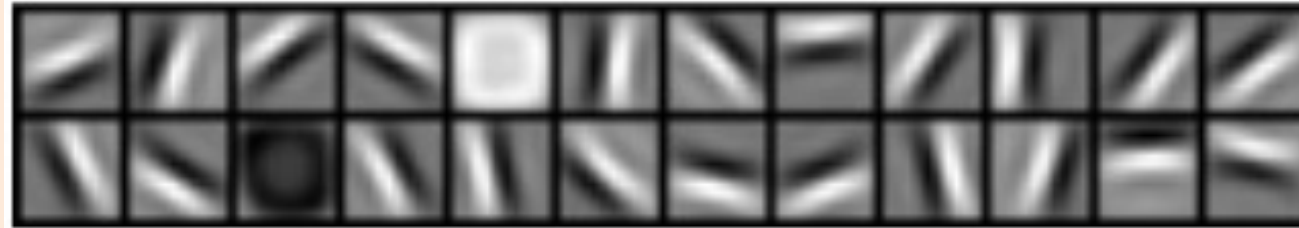
elephants

chairs



Cool Picture Motivation for Deep Learning

- First layer of z_i trained on 10 by 10 image patches:



} "Gabor filters"

- Visualization of second and third layers trained on specific objects:

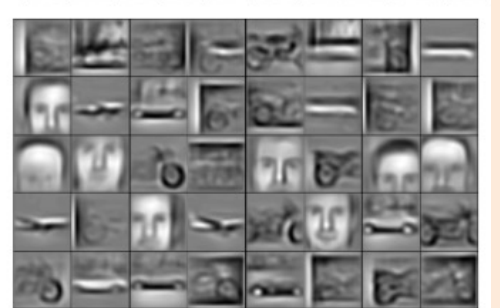
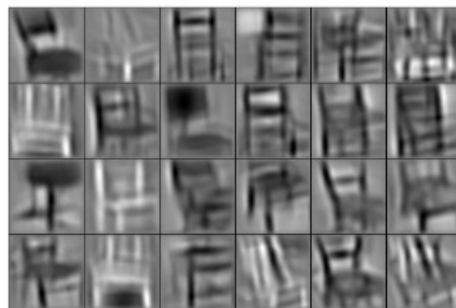
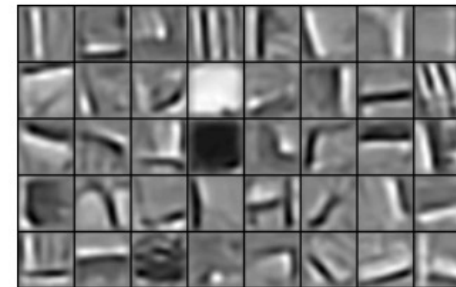
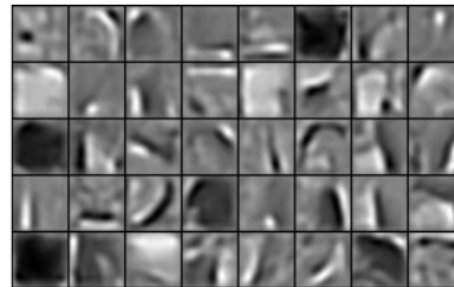
faces

cars

elephants

chairs

faces, cars, airplanes, motorbikes



DenseNet

- Variation on ResNets is “DenseNets”:
 - Each layer can see all the values from many previous layers.
 - Significantly reduces vanishing gradients.
 - May get same performance with fewer parameters/layers.

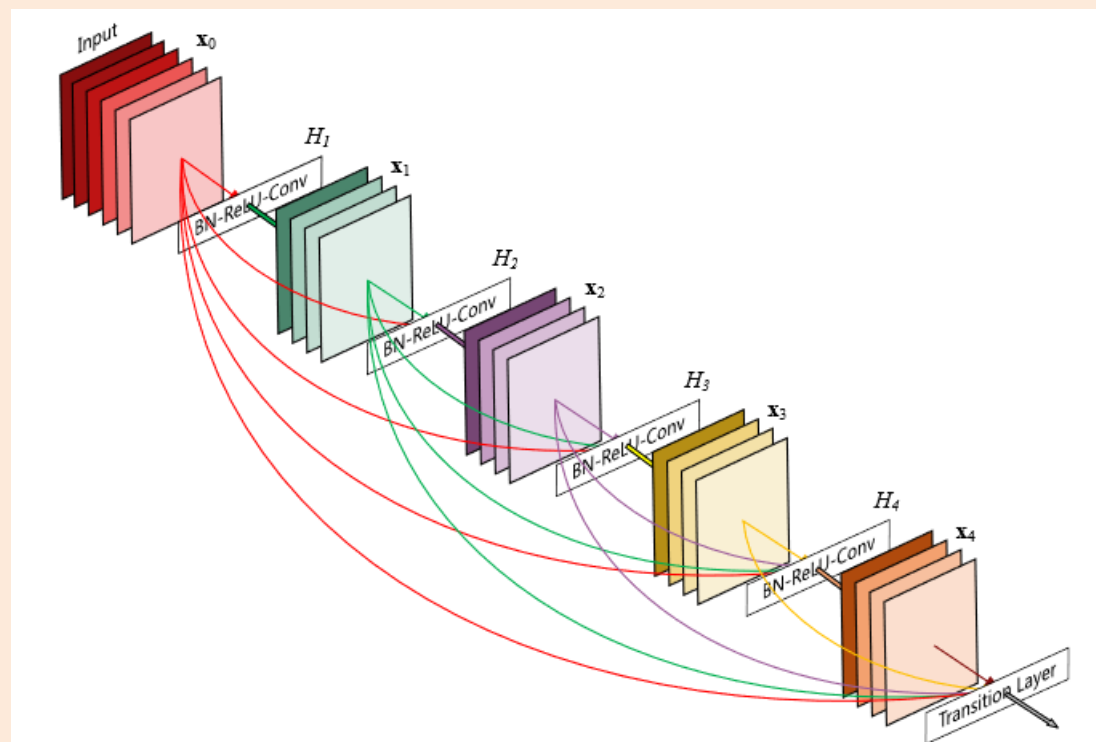


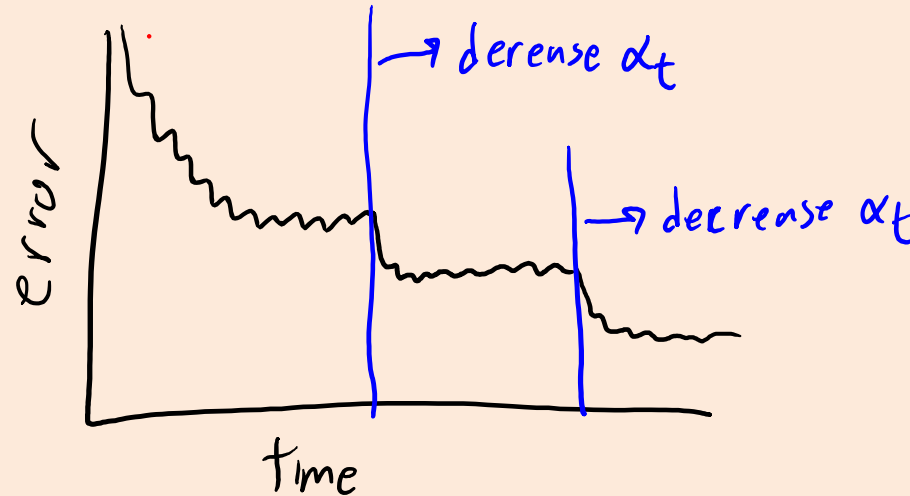
Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

Setting the Step-Size: Bottou Trick

- Automatic method to set step size is **Bottou trick**:
 1. Grab a small set of training examples (maybe 5% of total).
 2. Do a **binary search for a step size** that works well on them.
 3. Use this step size for a long time (or slowly decrease it from there).

Setting the Step-Size

- Stochastic gradient is **very sensitive to the step size** in deep models.
- Common approach: **manual “babysitting”** of the step-size.
 - Run SG for a while with a fixed step-size.
 - Occasionally measure error and plot progress:



- If error is not decreasing, decrease step-size.

Setting the Step-Size

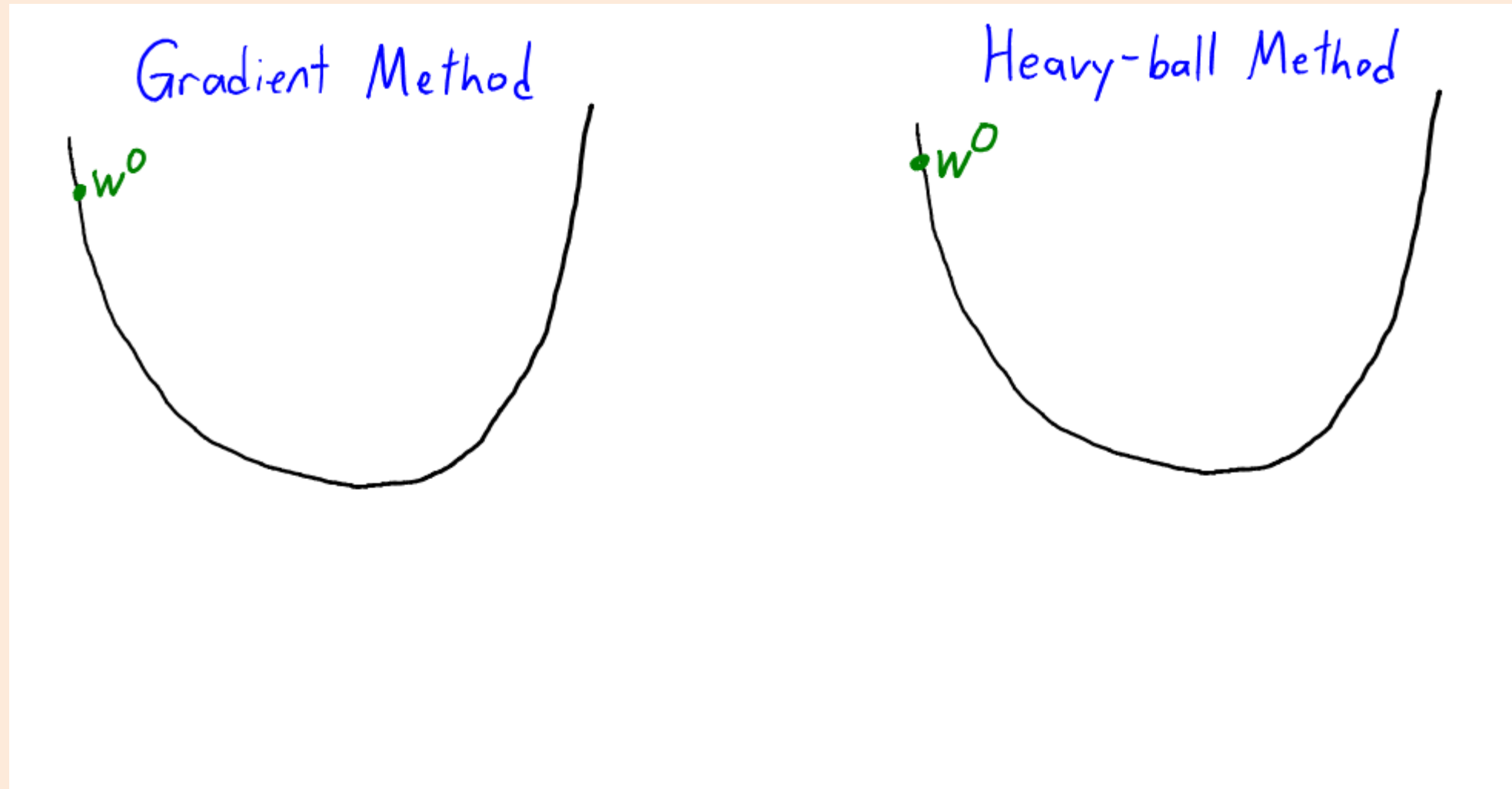
- Stochastic gradient is **very sensitive to the step size** in deep models.
- **Momentum** (stochastic version of “heavy-ball” algorithm):
 - Add term that moves in previous direction:

$$w^{t+1} = w^t - \alpha^t \nabla f_i(w^t) + \beta^t (w^t - w^{t-1})$$

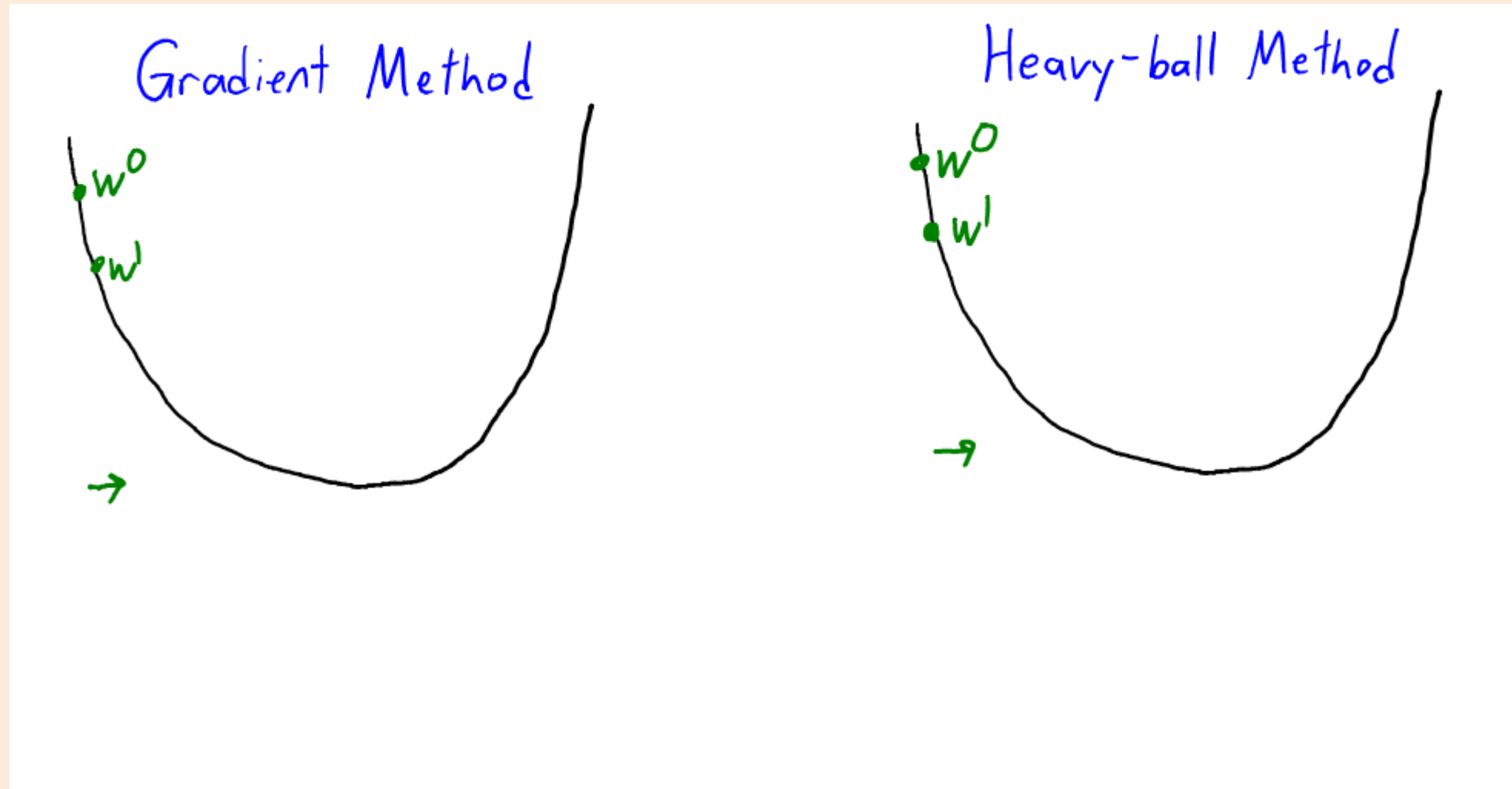
- Usually $\beta^t = 0.9$.

Keep going in the old direction

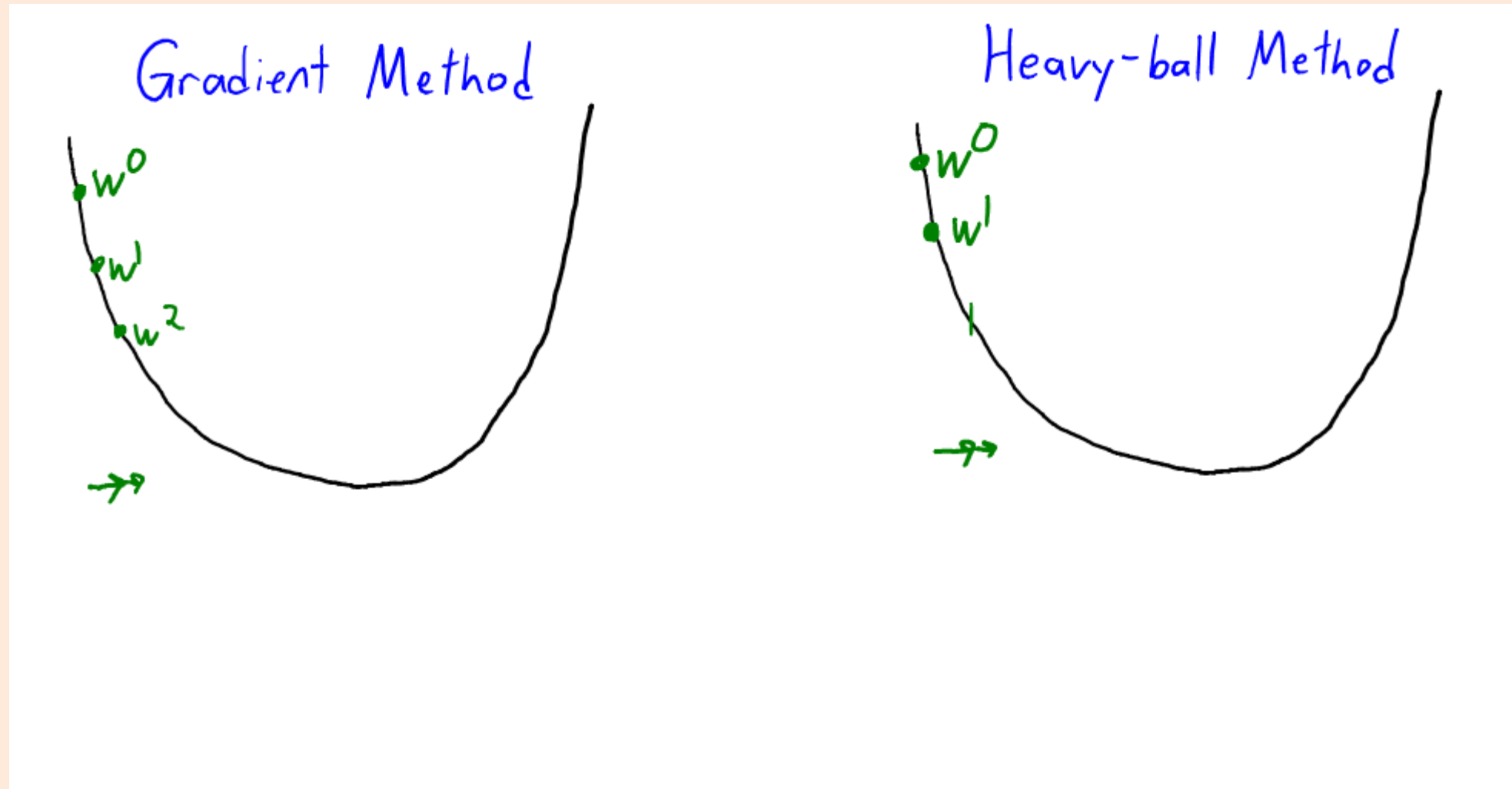
Gradient Descent vs. Heavy-Ball Method



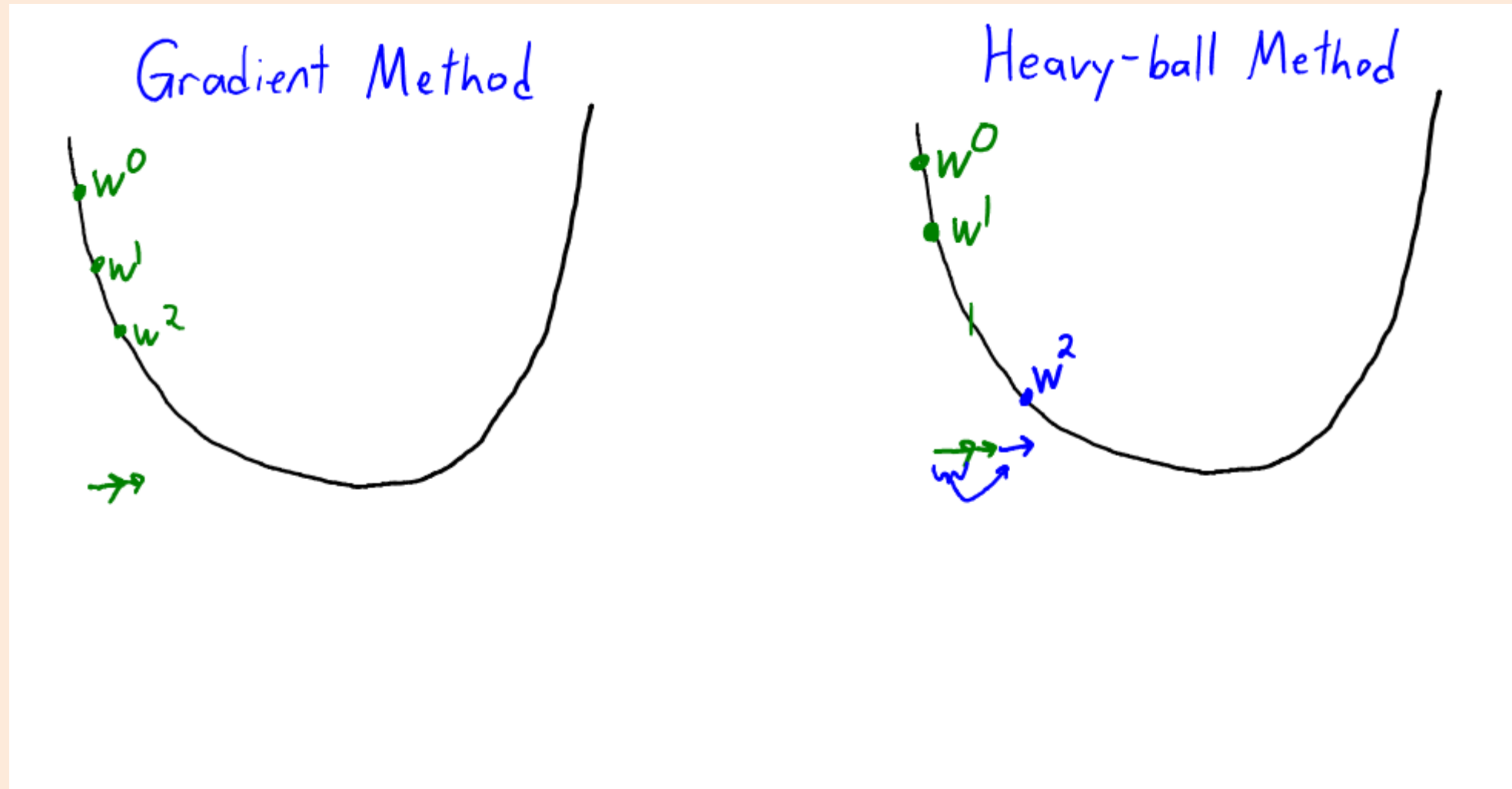
Gradient Descent vs. Heavy-Ball Method



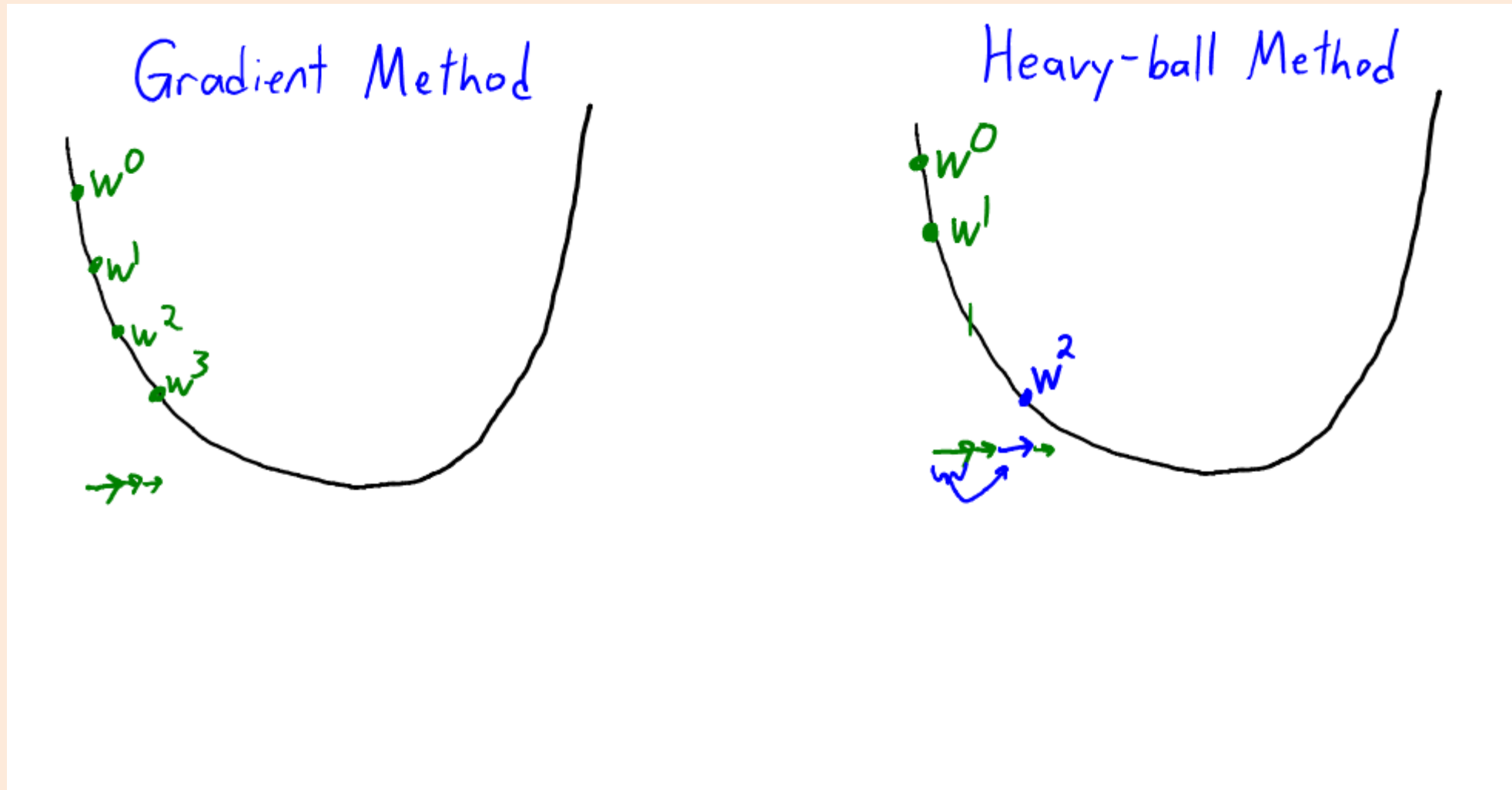
Gradient Descent vs. Heavy-Ball Method



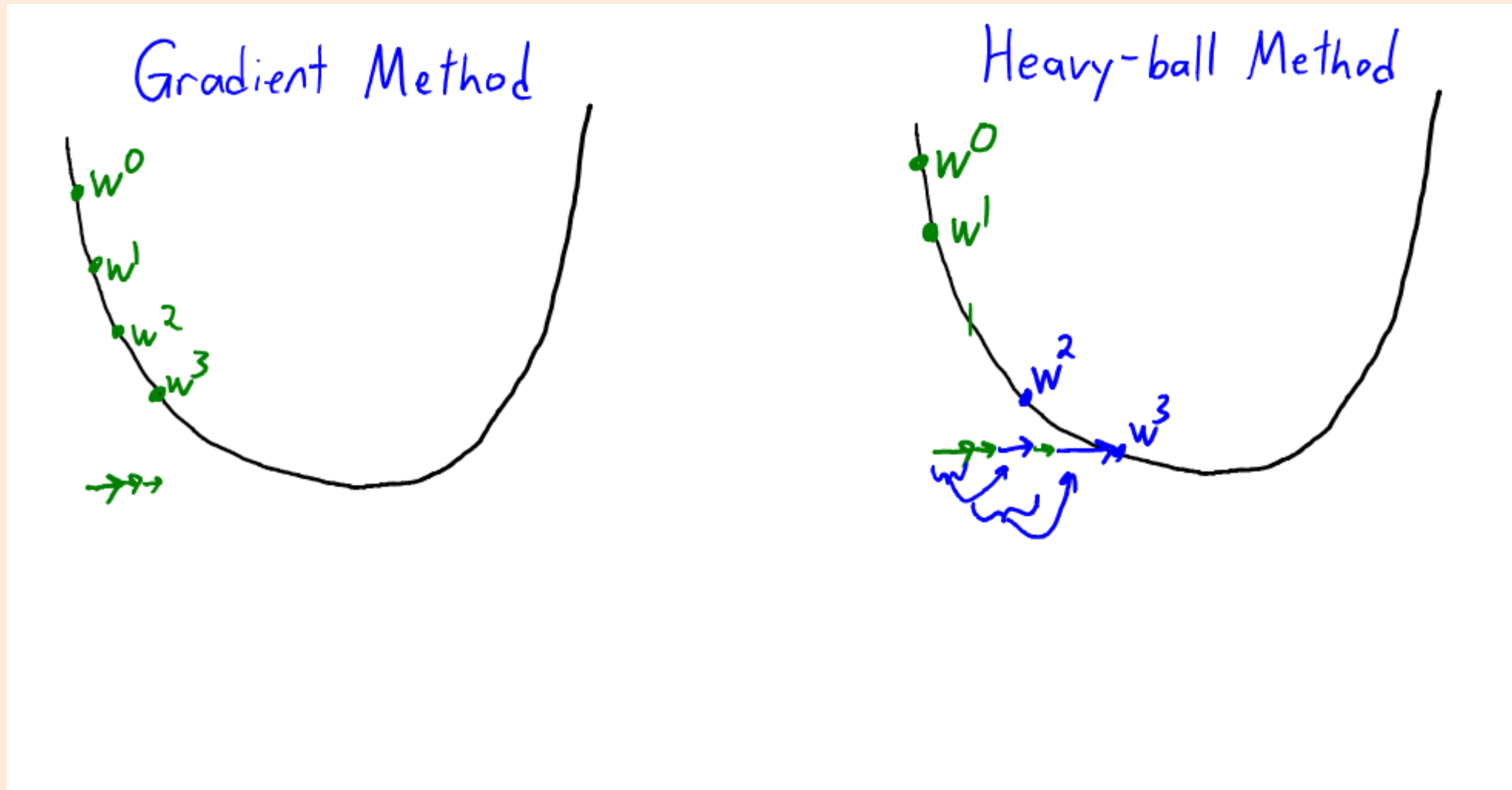
Gradient Descent vs. Heavy-Ball Method



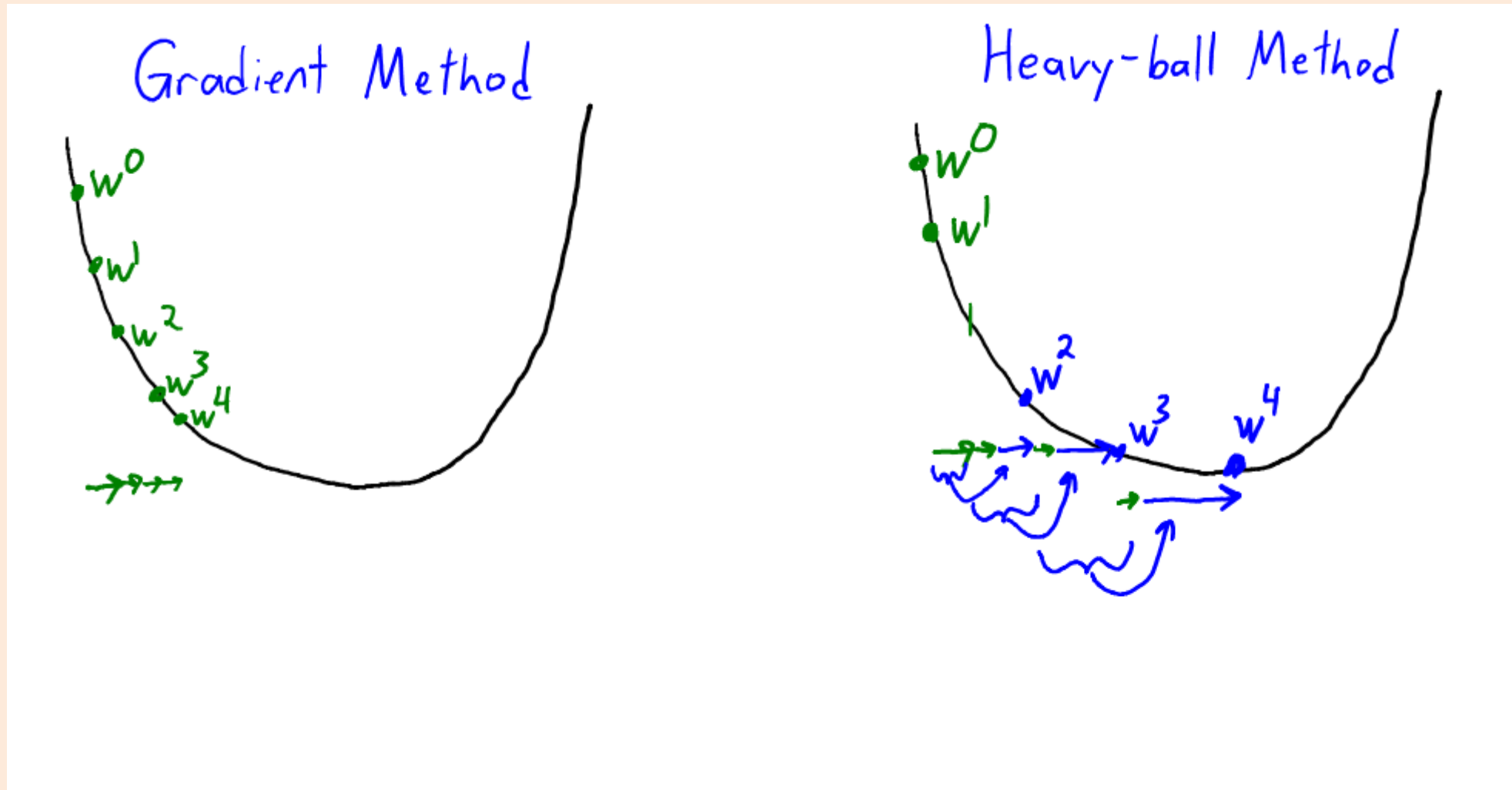
Gradient Descent vs. Heavy-Ball Method



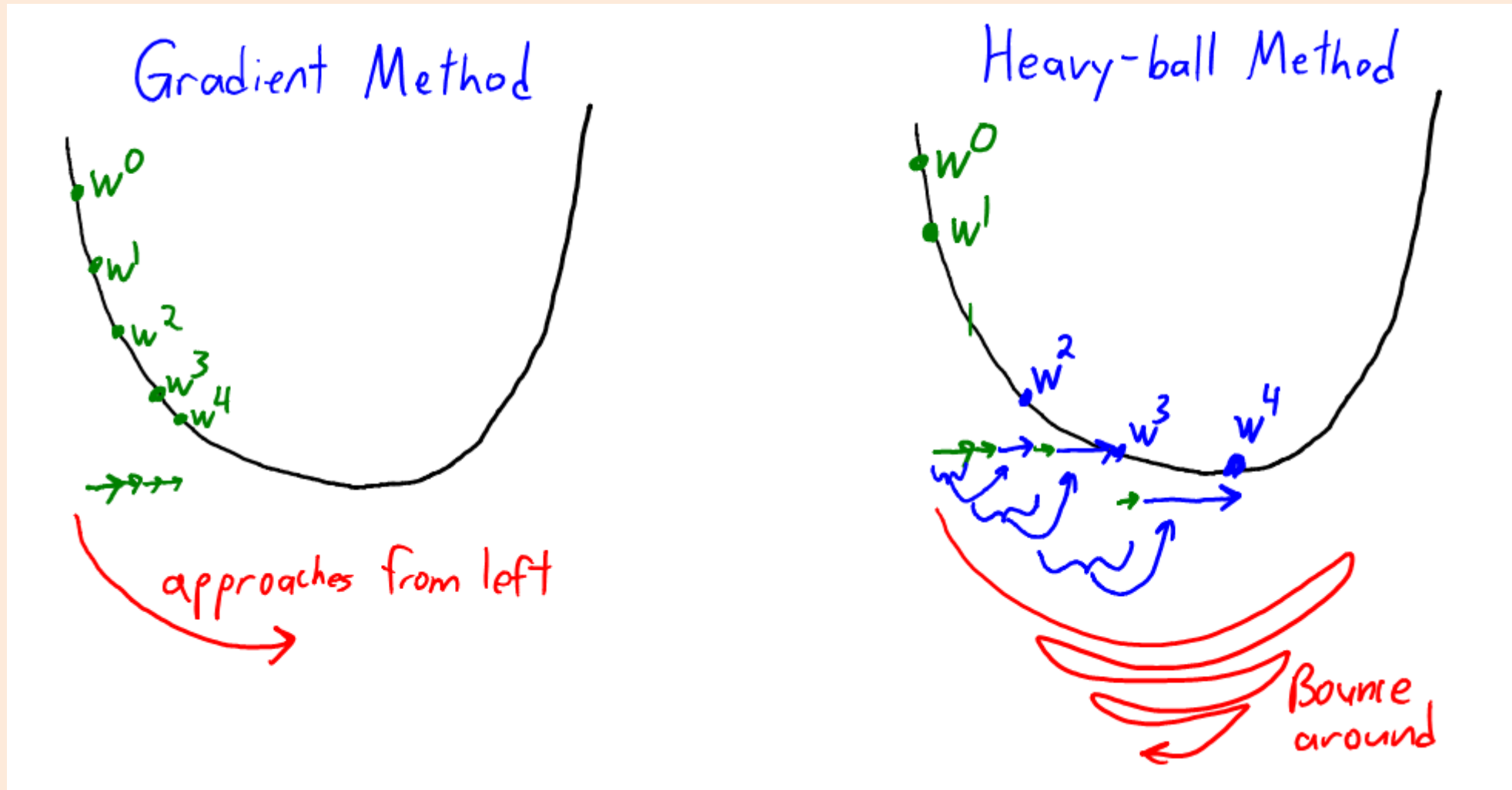
Gradient Descent vs. Heavy-Ball Method



Gradient Descent vs. Heavy-Ball Method



Gradient Descent vs. Heavy-Ball Method

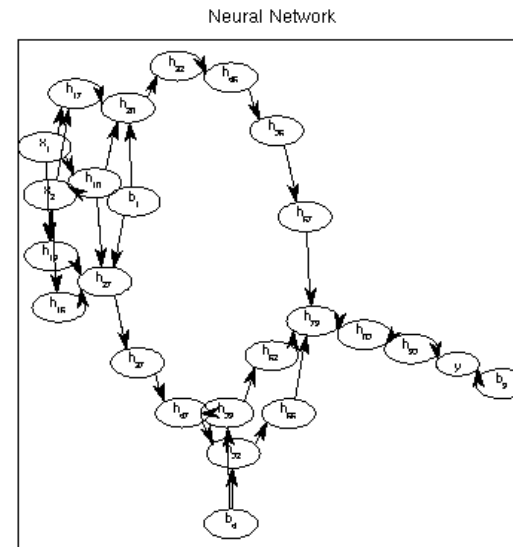
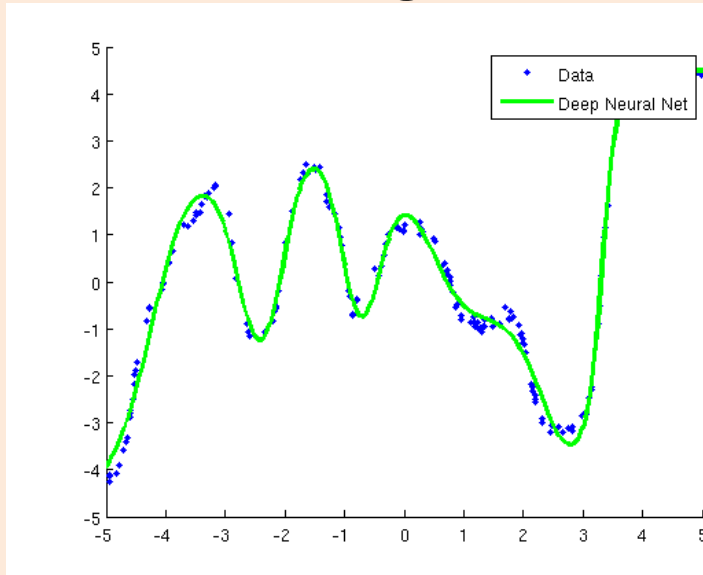


Standard Regularization

- Traditionally, we've added our usual **L2-regularizers**:

$$f(v, W^{(3)}, W^{(2)}, W^{(1)}) = \frac{1}{2} \sum_{i=1}^n (v^T h(W^{(3)} h(W^{(2)} h(W^{(1)} x_i))) - y_i)^2 + \frac{\lambda_4}{2} \|v\|^2 + \frac{\lambda_3}{2} \|W^{(3)}\|_F^2 + \frac{\lambda_2}{2} \|W^{(2)}\|_F^2 + \frac{\lambda_1}{2} \|W^{(1)}\|_F^2$$

- L2-regularization often called “**weight decay**” in this context.
 - Could also use L1-regularization: gives **sparse network**.



Standard Regularization

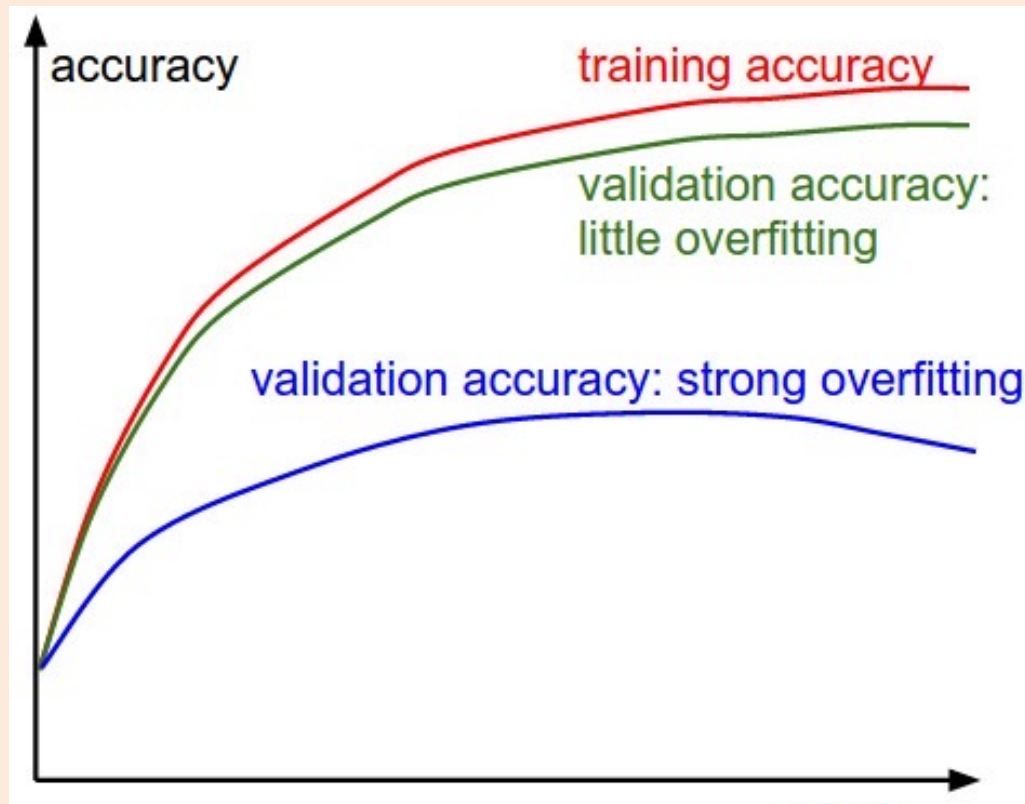
- Traditionally, we've added our usual **L2-regularizers**:

$$f(v, W^{(3)}, W^{(2)}, W^{(1)}) = \frac{1}{2} \sum_{i=1}^n (v^T h(W^{(3)} h(W^{(2)} h(W^{(1)} x_i))) - y_i)^2 + \frac{\lambda_4}{2} \|v\|^2 + \frac{\lambda_3}{2} \|W^{(3)}\|_F^2 + \frac{\lambda_2}{2} \|W^{(2)}\|_F^2 + \frac{\lambda_1}{2} \|W^{(1)}\|_F^2$$

- L2-regularization often called “**weight decay**” in this context.
 - Could also use L1-regularization: gives **sparse network**.
- Hyper-parameter** optimization gets **expensive**:
 - Try to optimize validation error in terms of $\lambda_1, \lambda_2, \lambda_3, \lambda_4$.
 - In addition to step-size, number of layers, size of layers, initialization.
- Recent result:
 - Adding a regularizer in this way **creates bad local optima**.

Early Stopping

- Another common type of regularization is “early stopping”:
 - Monitor the validation error as we run stochastic gradient.
 - Stop the algorithm if validation error starts increasing.

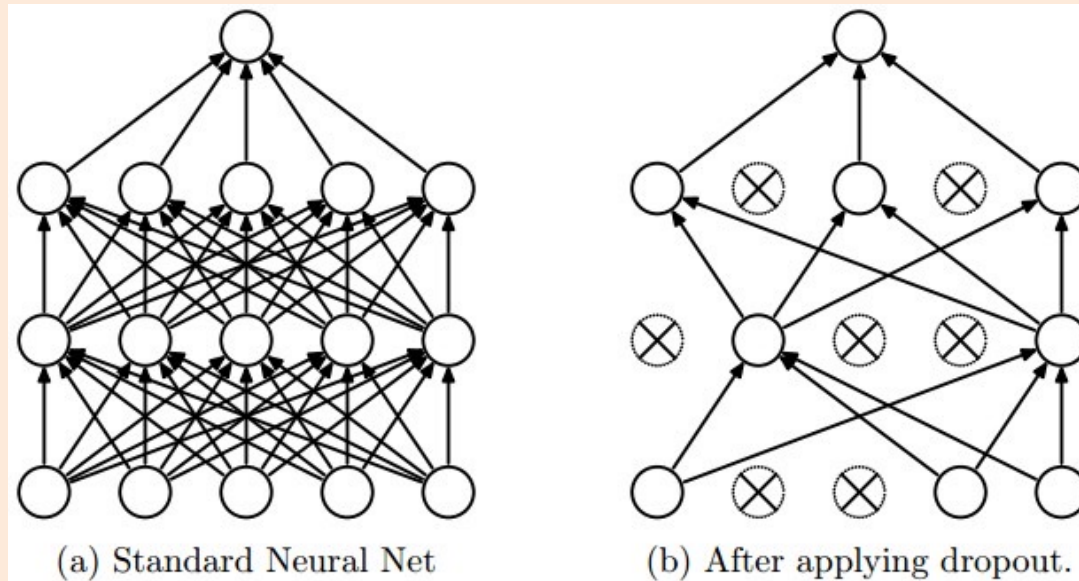


Unfortunately it might
look more like

↑ hopefully you don't
stop here.

Dropout

- **Dropout** is a more recent form of explicit regularization:
 - On each iteration, **randomly set some x_i and z_i to zero** (often use 50%).



- Adds **invariance to missing inputs or latent factors**
 - Encourages **distributed representation** rather than relying on specific z_i .
- Can be interpreted as an ensemble over networks with different parts missing.
- After a lot of success, dropout may already be going out of fashion.